

# Spenso

Typed tensors, symbolic structures, and executable tensor networks

---

## 1 Overview

Spenso represents and evaluates tensors with abstract indices. It separates a tensor's structure—representations, slots, dimensions, names, and index order—from its stored data. Dense and sparse storage, symbolic and parametric values, automatic index matching, and network execution build on that separation.

### Two questions, two layers

Tensor structure answers “which indices and representations does this object carry?” Tensor data answers “which values occupy that structure?” Contraction first matches compatible slots, then combines data. Keep these phases distinct when diagnosing a shape, duality, or contraction error.

### 1.1 Choose a task

- To verify slot duality with one numerical contraction, follow the [first-contraction tutorial](#) and the exact `Contract` reference.
- To assemble and execute several tensor/scalar nodes, use the [tensor-network manual](#) with the `Network` reference.
- To parse symbolic tensor functions or apply Dirac/color identities, review the [feature and interface guide](#), then continue with `Idenso` for representation-aware rewrites.

### 1.2 From tensors to networks

The core package supports three related levels of work:

- tensor structures describe slots and abstract indices;
- dense, sparse, numeric, symbolic, and parametric tensors attach data to those structures;
- tensor networks represent a collection of tensors and choose an execution or contraction strategy.

Networks can be assembled directly. With the `shadowing` feature, they can also be inferred from Symbolica expressions whose functions carry recognizable tensor structure. Parsing is not mere string conversion: representation initialization, index variance, names, and the tensor library determine the inferred network.

```
[dependencies]  
spenso = "0.6"
```

Enable shadowing only when Symbolica-backed parsing or symbolic operations are needed. It pulls in the Symbolica integration and corresponding Linnet support. The default core remains useful for typed tensors and contractions without that layer.

### 1.3 Companion packages

#### Component ownership

`spenso` owns the tensor model and execution engine. `spenso-macros` owns the `SimpleRepresentation` derive used to declare custom index representations. `spenso-hep-lib` owns concrete high-energy-physics tensor-library data such as gamma matrices and projectors. The `spynso3` adapter owns the Python community module. Their versions and feature gates are independent, so select and upgrade each component explicitly.

Spenso uses [Linnet](#) for the underlying network graph. [Idenso](#) is the symbolic-identity layer for Spenso-formatted Symbolica expressions. [GammaLoop](#) consumes these components inside a broader collider workflow.

The [Spenso README](#) gives a compact package summary. The Rust and Python API sections list the public modules, traits, feature gates, and import paths.

## 2 Tutorial

This tutorial performs one concrete tensor contraction in Rust. The example makes the tensor structure, dual index matching, storage, and numerical result visible—the same layers that a larger Spenso network coordinates automatically.

### 2.1 Prerequisites

Use Rust 1.85 or newer (Spenso uses Rust 2024 edition), then create a binary project:

```
cargo new spenso-first-contraction
cd spenso-first-contraction
cargo add spenso@0.6
```

This first contraction uses dense integer tensors and needs neither Symbolica nor the shadowing feature.

### 2.2 Contract one shared index

Replace `src/main.rs` with:

```
use spenso::{
    contraction::Contract,
    structure::{
        OrderedStructure, PermutedStructure,
        representation::{LibraryRep, Lorentz, RepName},
        slot::{DualSlotTo, IsAbstractSlot},
    },
    tensors::data::{DenseTensor, GetTensorData, SetTensorData},
};

fn main() {
    let rep = Lorentz {};
    let left_free = rep.new_slot(2, 0).to_lib();
    let right_free = rep.new_slot(2, 2).to_lib();
    let shared = rep.new_slot(2, 10).to_lib();

    let left_structure: OrderedStructure<LibraryRep> =
        PermutedStructure::from_iter([left_free, shared]).structure;
    let right_structure: OrderedStructure<LibraryRep> =
        PermutedStructure::from_iter([right_free, shared.dual()]).structure;

    let mut left = DenseTensor::<i32, _>::zero(left_structure);
    let mut right = DenseTensor::<i32, _>::zero(right_structure);
    left.set(&[0, 1], 2).unwrap();
    right.set(&[0, 1], 5).unwrap();

    let result = left.contract(&right).unwrap();
    assert_eq!(*result.get_ref([0, 0]).unwrap(), 10);
    println!("result structure: {}", result.structure);
    println!("result data: {:?}", result.data);
}
```

Run `cargo run`. Success means the assertion passes and the printed result contains 10 at the component selected by the two remaining free indices. Spenso matched `shared` with `shared.dual()`, summed that axis, and preserved `left_free` and `right_free` in the output structure. The contraction follows index identity and duality, not merely equal dimensions.

#### Verification scope and cost

The docs harness compiles and runs this Rust program; it syntax-checks the setup commands without creating a project or using the network. Success is the asserted component value 10 with two free output indices. The contraction is tiny; a clean external Cargo build can take minutes, while the program itself should finish in well under a second.

#### Structure is part of the value

The data vectors alone do not say which axes may contract. Preserve the `OrderedStructure` with the tensor, and inspect the resulting structure before interpreting a flat component offset. A dimension match with unrelated abstract indices produces an exterior product, not the contraction shown above.

## 2.3 From two tensors to a network

Pairwise Contract is the clearest first success. For a larger expression, place tensors in a `Network`, inspect its graph, choose or benchmark a contraction strategy, and execute it. Use `DataTensor` when individual nodes may be dense or sparse. Enable shadowing only when the network must mix concrete data with Symbolica-backed parametric tensors.

## 2.4 Troubleshooting and next steps

- If contract leaves both shared-looking axes free, print their slots and check that one is the dual of the other; dimensions alone are insufficient.
- If set rejects a component, compare the coordinate count and each coordinate with the structure's rank and dimensions.
- If a sparse contraction reports an empty input, distinguish an intentionally zero tensor from a tensor whose nonzero entries were never populated.
- Continue with the tensor-structures and networks manual before selecting an automatic contraction strategy.

## 3 Tensor data, contraction, and network execution

Spenso keeps tensor structure and tensor data separate so the same structural operations work with dense, sparse, symbolic, or parametric storage. A structure supplies ordered slots, representations, dimensions, and abstract indices. Data supplies values and a storage-specific access strategy. Contraction first establishes structural compatibility and only then chooses a data operation.

### 3.1 Dense, sparse, and parametric data

Dense storage is appropriate when most components are populated and a predictable contiguous layout matters. Sparse storage is appropriate when zero structure dominates, but contraction cost depends on index overlap and lookup behavior rather than only the number of stored values. Parametric and symbolic tensors defer numeric substitution; keep their parameter environment with the tensor when serializing or moving a calculation between stages.

Conversions must preserve the structure's slot order. A tensor with the same dimensions but a different representation or variance is not interchangeable until an explicit structural operation establishes that relationship.

## 3.2 Contraction and planning

Pairwise contraction matches compatible dual slots, determines the output structure, and then contracts the selected data backends. A tensor network generalizes this to many nodes connected through abstract indices. Planning selects a contraction order; execution applies that plan to the current data.

The cheapest-looking local pair is not always the cheapest complete plan. Intermediate tensor dimensions, sparsity, symbolic expression growth, and reusable subexpressions can dominate. Record the planner/configuration with performance results and invalidate a stored plan when the network structure changes.

### Plan structure, execute data

A plan is meaningful for the network structure it was derived from. Replacing values without changing slots can reuse it; adding a tensor, changing a representation, or rewiring an index requires validation or replanning.

## 3.3 Execute a small tensor/scalar network

This self-contained network computes  $2a + 3b$  for two equally structured tensors. The dummy libraries make the execution boundary explicit: every tensor is already concrete and any unresolved function key is an error.

```
use spenso::{
    network::{
        ExecutionResult, Network, Sequential, SmallestDegree,
        library::{DummyKey, DummyLibrary, DummyLibraryTensor, panicing::ErroringLibrary},
        store::NetworkStore,
    },
    structure::{
        OrderedStructure,
        representation::{Euclidean, RepName},
    },
    tensors::data::DenseTensor,
};

fn main() {
    type Tensor = DenseTensor<f64, OrderedStructure<Euclidean>>;
    type Store = NetworkStore<Tensor, f64>;
    type Net = Network<Store, DummyKey, DummyKey>;
    type LibraryTensor = DummyLibraryTensor<Tensor>;

    let structure = OrderedStructure::new(vec![Euclidean {}.new_slot(2, 1)]).structure;
    let a = DenseTensor::from_data(vec![1.0, 2.0], structure.clone()).unwrap();
    let b = DenseTensor::from_data(vec![3.0, 4.0], structure).unwrap();
    let tensors = DummyLibrary::new();
    let functions = ErroringLibrary::new();

    let mut network =
        Net::from_scalar(2.0) * Net::from_tensor(a) + Net::from_scalar(3.0) *
Net::from_tensor(b);
    network
        .execute::<Sequential, SmallestDegree, LibraryTensor, _, _>(&tensors, &functions)
        .unwrap();

    let ExecutionResult::Val(result) = network.result_tensor::<LibraryTensor,
_>(&tensors).unwrap()
    else {
        panic!("expected a concrete tensor result");
    };
    assert_eq!(result.data, vec![11.0, 16.0]);
}
```

The output invariant is the component vector `[11.0, 16.0]`; the original rank-one structure must also remain attached to the result. The documentation harness compiles the complete program; run it to execute the assertion in a provisioned Symbolica environment. See the exact `Network`, `DenseTensor`, and `pairwise contraction` references.

#### Interpret execution failures by layer

`from_data` errors are rank/dimension or storage-length mismatches. An unresolved function-key error means the network contains a symbolic library node despite the concrete-only setup. A different value with the same shape points to scalar placement or node arithmetic; a different shape points to changed slots or unintended contraction, which requires replanning.

### 3.4 Symbolica shadowing

With shadowing, Spenso recognizes registered tensor functions inside Symbolica expressions and builds a network whose nodes shadow those subexpressions. Initialization of representations and the tensor library must happen before parsing. Unknown functions remain symbolic rather than silently acquiring tensor semantics.

Shadowing is useful for extracting a contraction problem while retaining a reversible link to the source expression. Keep the replacement map until the computed result has been substituted back. For rewrite identities and index cooking, use `Idenso` Spenso itself owns storage, contraction, and execution.

### 3.5 Custom representations and the HEP library

`SimpleRepresentation` derives the repetitive representation plumbing for a user enum. The derive input still defines domain semantics: duality, dimension, slot compatibility, and any Symbolica tags must agree. The resulting representations implement the ordinary Rust traits used by the rest of Spenso; the `spenso-macros` API reference lists the supported derive attributes.

`spenso-hep-lib` registers concrete high-energy-physics structures such as gamma matrices and projectors. Treat that library as shared domain data. `GammaLoop` uses the same definitions.

#### Graph and tensor ownership

`Linnet` owns connectivity, cuts, cycles, and graph mutation. Spenso owns which connected slots contract and how a plan is executed. Keeping that boundary explicit prevents a graph rewrite from being mistaken for a tensor-algebra identity.

## 4 Rust, macros, and Python APIs

### 4.1 Core Rust package

The `spenso` crate organizes its public surface into `structure`, `tensors`, `contraction`, `network`, `iterators`, and `algebra`. The important abstractions form a progression:

- `TensorStructure` and related traits describe slots, names, and contraction compatibility;
- `DenseTensor`, `SparseTensor`, and the heterogeneous tensor enums own storage;
- contraction traits perform pairwise or multi-tensor operations;
- network stores and libraries bind symbolic tensor names to concrete data and execute a graph.

Trait implementations and generic constraints determine which combinations of structure and data support an operation. Consult the [curated Rust API](#) when a method is unavailable for a particular tensor type, then follow its `Rustdoc` link for exhaustive generic detail. The shadowing API and symbolic parallelism controls require their corresponding Cargo features.

## 4.2 Proc macros and HEP data

spenso-macros is a separate proc-macro crate. Its SimpleRepresentation derive generates the representation and duality boilerplate used by Spenso index types. A declaration supplies a symbolic name and chooses either self\_dual or a dual\_name:

```
use spenso_macros::SimpleRepresentation;
use spenso::structure::representation::RepName;

#[derive(SimpleRepresentation)]
#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash, PartialOrd, Ord, Default)]
#[representation(name = "flavor", dual_name = "AntiFlavor")]
struct Flavor {}
```

The spenso-macros card in the [Rust reference](#) lists the derive's helper attributes, allowed targets, and feature conditions. Macro expansion happens at compile time and produces ordinary Rust implementations.

spenso-hep-lib supplies domain data and tensor-library construction for high-energy physics. It is intentionally separate from the generic core. Users who need gamma matrices or physics projectors add that package; generic Spenso users do not inherit those conventions implicitly.

## 4.3 Python community module

### An adapter, not a Spenso wheel

Python users import `symbolica.community.spenso`. The implementation is the `spynso3` Rust adapter and is distributed through the Symbolica community-module mechanism. Enabling Spenso's Rust python feature only enables conversion interoperability; it does not create a standalone importable spenso Python distribution.

For a released Python assembly that includes Spenso, install the provider's Symbolica distribution with `python -m pip install symbolica`, then verify `python -c "import symbolica.community.spenso"`. Module availability follows the Symbolica assembly version; a local Spenso source checkout does not add the module to an installed wheel. Source embedders must add `spynso3` to the external `symbolica-community` assembly and invoke its `SymbolicaCommunityModule` registration while building that extension; building the Rust crate alone does not inject the module into another Symbolica wheel.

```
from symbolica.community.spenso import (
    Representation,
    Tensor,
    TensorIndices,
)

rep = Representation.euc(2)
indices = TensorIndices(rep("i"), rep("j"))
identity = Tensor.dense(indices, [1.0, 0.0, 0.0, 1.0])
```

The structured [Python reference](#) includes tensor and structure types, tensor networks, execution modes, tensor libraries, expression evaluators, and symbolic-parallelism controls. Its signatures can differ from the generic Rust API because `spynso3` provides the Python-specific conversions and defaults.

Source starting points are the core crate, the derive crate, the HEP library, and the Python adapter.

## 5 Releases and change history

### Version coverage

This version of Spenso is 0.6.0, but the published changelog currently ends at 0.5.6. Release notes for 0.6.0 are not yet available, so the history below is incomplete for that version. `spenso-macros`, `spenso-hep-lib`, and `spynso3` have independent versions and changelogs.

When upgrading, inspect the history for tensor structure, contraction, network parsing, Symbolica integration, and Python API or signature changes. Also check each companion component used by the application: a core Spenso release does not imply a release of the macro, HEP, or Python adapter package.

For reproducible upgrades, record the versions of Spenso and every companion component used by the application. Do not assume that changes between 0.5.6 and 0.6.0 are fully represented in the available notes.

The core Spenso changelog is followed by separately routed histories for `spenso-macros`, `spenso-hep-lib`, and `spynso3`. Each page links back to its canonical Typst source.

## 6 Spenso macros release history

The procedural macros are versioned independently from Spenso. Their canonical history remains the `spenso-macros` changelog source. This rendered copy integrates it with the manual, navigation, search, and product PDF.

All notable changes to this project will be documented in this file.

The format is based on [Keep a Changelog](#), and this project adheres to [Semantic Versioning](#).

### 7.1 Unreleased

#### 7.2 0.3.2 - 2026-01-08

##### 7.2.1 Fixed

- fix metric initialization and update to symbolica 1.2

##### 7.2.2 Other

- update linnet

#### 7.3 0.3.1 - 2025-12-15

##### 7.3.1 Other

- Update symbolica to 1.1.0
- update linnet
- update linnet
- add readmes
- update linnet and make classattr to classmethods

#### 7.4 0.2.1 - 2025-08-18

##### 7.4.1 Other

- update linnet to crates
- update linnet
- update linnet
- update linnet

## 8 Spenso HEP library release history

The HEP helper library is versioned independently from Spenso. Its canonical history remains the [spenso-hep-lib changelog source](#). This rendered copy integrates it with the manual, navigation, search, and product PDF.

All notable changes to this project will be documented in this file.

The format is based on [Keep a Changelog](#), and this project adheres to [Semantic Versioning](#).

### 9.1 Unreleased

#### 9.2 0.1.7 - 2026-01-14

##### 9.2.1 Other

- update versions

#### 9.3 0.1.6 - 2026-01-08

##### 9.3.1 Fixed

- fix python\_stub\_gen feature
- fix metric initialization and update to symbolica 1.2

#### 9.4 0.1.5 - 2025-12-15

##### 9.4.1 Fixed

- fix warnings
- fix add\_assign with sparse tensors

##### 9.4.2 Other

- Update symbolica to 1.1.0
- update linnet
- add readmes
- Release spenso-macros 0.3
- update to symbolica 1.0
- update sympolica to 0.20
- Properly precontract scalars
- Refactor coefficient conversions to support generic float types
- working pow and fun execution
- working pow execution with wrapping
- add function generic key

#### 9.5 0.1.3 - 2025-08-18

##### 9.5.1 Other

- all tests pass
- update linnet



## 9.6 0.1.2 - 2025-07-15

### 9.6.1 Fixed

- fix gamma algebra again

### 9.6.2 Other

- update versions

## 9.7 0.1.1 - 2025-07-15

### 9.7.1 Fixed

- fix gamma algebra~

### 9.7.2 Other

- update release action
- allow arbitrary arguments for to dots

## 10 Spynso3 release history

The Python adapter is versioned independently from Spenso. Its canonical history remains the [spynso3 changelog source](#). This rendered copy integrates it with the manual, navigation, search, and product PDF.

All notable changes to this project will be documented in this file.

The format is based on [Keep a Changelog](#), and this project adheres to [Semantic Versioning](#).

## 11.1 Unreleased

## 11.2 0.1.2 - 2026-01-14

### 11.2.1 Other

- update versions

## 11.3 0.1.1 - 2025-12-15

### 11.3.1 Fixed

- fix sum normalization in network parsing
- fix warnings
- fix add\_assign with sparse tensors
- fix projp
- fix param docs and iter return type
- fix pyo3\_stubs

### 11.3.2 Other

- Update implementation to pyo3-stub-gen 0.17
- Update symbolica to 1.1.0
- update linnet
- add readmes
- update to 0.17 pyo3\_stub\_gen

- update linnet and make classattr to classmethods
- move initialize to api
- Release spenso-macros 0.3
- update to symbolica 1.0
- update sympolica to 0.20
- Properly precontract scalars
- Add back pyo3-stub-gen-derive
- update to current dev symbolica and add pattern for pslash + m conjugate
- first try conj
- update python api
- add function generic key
- again
- return any because python is dumb
- update `__iter__` method
- use numpy doc format
- add overloaded pyi stub gen

## API reference

The packages available in this version are listed below. Follow an item link for its complete language reference.

### spenso

Spensor's supported tensor and network API. The scope is *intentionally* ordered so foundational tensor types precede planning and execution.

#### Tensor structure

Describes ordered slots, dimensions, representations, and contraction compatibility independently of data.

Describes ordered slots, dimensions, representations, and contraction compatibility independently of data.

```
pub trait TensorStructure
{
  type Slot : IsAbstractSlot + DualSlotTo < Dual = Self :: Slot > ; type
  Indexed : TensorStructure < Indexed = Self :: Indexed, Slot = Self :: Slot
  > ; fn
  reindex(self, indices : & [< Self :: Slot as IsAbstractSlot > :: Aind],)
  -> Result < PermutedStructure < Self :: Indexed > , StructureError > ; fn
  dual(self) -> Self; fn string_rep(& self) -> String
  {
    self.external_structure_iter().map(| s | s.to_string()).collect :: <
    Vec < _ > > ().join("")
  } fn external_structure_iter(& self) -> impl Iterator < Item = Self ::
  Slot > ; fn external_dims_iter(& self) -> impl Iterator < Item = Dimension
  > ; fn external_reps_iter(& self,) -> impl Iterator < Item =
  Representation < < Self :: Slot as IsAbstractSlot > :: R > > ; fn
  external_indices_iter(& self) -> impl Iterator < Item = < Self :: Slot as
  IsAbstractSlot > :: Aind > ; fn
  get_aind(& self, i : impl Into < SlotIndex >) -> Option < < Self :: Slot
  as IsAbstractSlot > :: Aind > ; fn
  get_rep(& self, i : impl Into < SlotIndex > ,) -> Option < Representation
  < < Self :: Slot as IsAbstractSlot > :: R > > ; fn
  get_dim(& self, i : impl Into < SlotIndex >) -> Option < Dimension > ; fn
  get_slot(& self, i : impl Into < SlotIndex >) -> Option < Self :: Slot > ;
  fn order(& self) -> usize;
  #[doc =
  " returns the list of slots that are the external indices of the tensor"]
  fn external_structure(& self) -> Vec < Self :: Slot >
  { self.external_structure_iter().collect() } fn to_shell(self) ->
  TensorShell < Self > where Self : Sized, { TensorShell :: new(self) } fn
  contains_matching(& self, slot : & Self :: Slot) -> bool
  { self.external_structure_iter().any(| s | s.matches(slot)) } fn
  external_reps(& self) -> Vec < Representation < < Self :: Slot as
  IsAbstractSlot > :: R > > { self.external_reps_iter().collect() } fn
  external_indices(& self) -> Vec < < Self :: Slot as IsAbstractSlot > ::
  Aind > { self.external_indices_iter().collect() }
  #[doc =
  " checks if the tensor has the same exact structure as another tensor"] fn
  same_content(& self, other : & Self) -> bool { self.same_external(other) }
  #[doc =
  " Given two [`TensorStructure`]s, returns the index of the first matching slot in
  each external index list, along with a boolean indicating if there is a single
  match"]
```

```

fn match_index(& self, other : & Self) -> Option < (bool, usize, usize) >
{
    let posmap =
    self.external_structure_iter().enumerate().map(| (i, slot) |
    (slot, i)).collect :: < AHashMap < _, _ > > (); let mut first_pair :
    Option < (usize, usize) > = None; for (j, slot) in
    other.external_structure_iter().enumerate()
    {
        if let Some(& i) = posmap.get(& slot.dual())
        {
            if let Some((i, j)) = first_pair
            { return Some((false, i, j)); } first_pair = Some((i, j));
        }
    } first_pair.map(| (i, j) | (true, i, j))
}
#[doc =
" Given two [`TensorStructure`]s, returns the index of the first matching slot in
each external index list"]
fn match_indices(& self, other : & Self) -> Option <
(Permutation, Vec < bool > , Vec < bool >) >
{
    let mut self_matches = vec! [false; self.order()]; let mut perm = Vec
:: new(); let mut other_matches = vec! [false; other.order()]; let
posmap =
self.external_structure_iter().enumerate().map(| (i, slot) |
(slot, i)).collect :: < AHashMap < _, _ > > (); for (j, slot_other) in
other.external_structure_iter().enumerate()
{
    if let Some(& i) = posmap.get(& slot_other.dual())
    {
        self_matches [i] = true; other_matches [j] = true;
        perm.push(i);
    }
} if perm.is_empty() { None } else
{
    let p : Permutation = Permutation :: sort(& perm);
    Some((p, self_matches, other_matches))
}
} #[doc = " Identify the repeated slots in the external index list"] fn
traces(& self) -> Vec < [usize; 2] >
{
    let mut positions : HashMap < & Self as TensorStructure > :: Slot, Vec
< usize > > = HashMap :: new(); for (index, key) in
self.external_structure_iter().enumerate()
{
    if let Some(v) = positions.get_mut(& key.dual())
    { v.push(index); } else { positions.insert(key, vec! [index]); }
}
positions.into_values().filter_map(| indices |
{
    if indices.len() == 2 { Some([indices [0], indices [1]]) } else
    { None }
}).collect()
}
#[doc =
" yields the (outwards facing) shape of the tensor as a list of dimensions"]

```

```

fn shape(& self) -> Vec < Dimension >
{ self.external_dims_iter().collect() } fn reps(& self) -> Vec <
Representation < < Self :: Slot as IsAbstractSlot > :: R > >
{ self.external_reps_iter().collect() }
#[doc = " checks if externally, the two tensors are the same"] fn
same_external(& self, other : & Self) -> bool
{
    let set1 : HashSet < _ > = self.external_structure_iter().collect();
    let set2 : HashSet < _ > = other.external_structure_iter().collect();
    set1 == set2
}
#[doc =
" find the permutation of the external indices that would make the two tensors
the same. Applying the permutation to other should make it the same as self"]
fn find_permutation(& self, other : & Self) -> Result < Permutation >
{
    if self.order() != other.order()
    {
        return
        Err(eyre!
            ("Mismatched order: {} vs {}", self.order(), other.order()));
    } let other_structure = other.external_structure(); let self_structure
= self.external_structure(); let other_sort = Permutation ::
sort(& other_structure); let self_sort = Permutation ::
sort(& self_structure); if other_sort.apply_slice(& other_structure)
== self_sort.apply_slice(& self_structure)
{ Ok(other_sort.compose(& self_sort.inverse())) } else
{ Err(eyre! ("Mismatched structure")) }
} #[doc = " yields the strides of the tensor in column major order"] fn
strides_column_major(& self) -> Result < Vec < usize > >
{
    let mut strides : Vec < usize > = vec! [1; self.order()]; if
self.order() == 0 { return Ok(strides); } for i in 0 .. self.order() -
1
    {
        strides [i + 1] = strides [i] * usize ::
try_from(self.shape() [i]) ? ;
    } Ok(strides)
} #[doc = " yields the strides of the tensor in row major order"] fn
strides_row_major(& self) -> Result < Vec < usize > >
{
    let mut strides = vec! [1; self.order()]; if self.order() == 0
{ return Ok(strides); } for i in (0 .. self.order() - 1).rev()
    {
        strides [i] = strides [i + 1] * usize ::
try_from(self.shape() [i + 1]) ? ;
    } Ok(strides)
} #[doc = " By default, the strides are row major"] fn strides(& self) ->
Result < Vec < usize > > { self.strides_row_major() }
#[doc =
" Verifies that the list of indices provided are valid for the tensor"]
#[doc = ""] #[doc = " # Errors"] #[doc = ""]
#[doc =
" `Mismatched order` = if the length of the indices is different from the order
of the tensor,"]
#[doc = ""]

```

```

#[doc =
" `Index out of bounds` = if the index is out of bounds for the dimension of that
index"]
#[doc = ""] fn verify_indices < C : AsRef < [ConcreteIndex] > >
(& self, indices : C) -> Result < () >
{
    if indices.as_ref().len() != self.order()
    {
        return
        Err(eyre!
            ("Mismatched order: {} indices, vs order {}",
            indices.as_ref().len(), self.order()));
    } for (i, dim_len) in
self.external_structure_iter().map(| slot | slot.dim()).enumerate()
{
    if indices.as_ref() [i] >= usize :: try_from(dim_len) ?
    {
        return
        Err(eyre!
            ("Index {} out of bounds for dimension {} of size {}",
            indices.as_ref()[i], i, usize::try_from(dim_len)?));
    }
} Ok(())
} #[doc = " yields the flat index of the tensor given a list of indices"]
#[doc = ""] #[doc = " # Errors"] #[doc = ""]
#[doc = " Same as [`Self::verify_indices`]"] fn flat_index < C : AsRef <
[ConcreteIndex] > > (& self, indices : C) -> Result < FlatIndex >
{
    let strides = self.strides() ? ; self.verify_indices(& indices) ? ;
    let mut idx = 0; for (i, & index) in
indices.as_ref().iter().enumerate() { idx += index * strides [i]; }
    Ok(idx.into())
} #[doc = " yields the expanded index of the tensor given a flat index"]
#[doc = ""] #[doc = " # Errors"] #[doc = ""]
#[doc =
" `Index out of bounds` = if the flat index is out of bounds for the tensor"]
fn expanded_index(& self, flat_index : FlatIndex) -> Result <
ExpandedIndex >
{
    let mut indices = vec! []; let mut index : usize = flat_index.into();
    for & stride in & self.strides() ?
    { indices.push(index / stride); index %= stride; } if usize ::
from(flat_index) < self.size() ? { Ok(indices.into()) } else
    { Err(eyre! ("Index {flat_index} out of bounds")) }
} fn co_expanded_index(& self, flat_index : FlatIndex) -> Result <
DualConciousExpandedIndex >
{
    let mut indices = vec! []; for (r, i) in
self.external_reps_iter().zip(self.expanded_index(flat_index) ?
.iter())
{
    if r.rep.is_base() && r.rep.is_dual()
    { indices.push(DualConciousIndex :: SelfDual(* i)); } else if
r.rep.is_base() { indices.push(DualConciousIndex :: Up(* i)); }
    else { indices.push(DualConciousIndex :: Down(* i)); }
} Ok(indices.into())
}

```

```

    } #[doc = " yields an iterator over the indices of the tensor"] fn
    index_iter(& self) -> TensorStructureIndexIterator < '_, Self > where Self
    : Sized, { TensorStructureIndexIterator :: new(self) }
    #[doc = " if the tensor has no (external) indices, it is a scalar"] fn
    is_scalar(& self) -> bool { self.order() == 0 } fn
    is_fully_self_dual(& self) -> bool;
    #[doc =
    " yields the size of the tensor, i.e. the product of the dimensions. This is the
    length of the vector of the data in a dense tensor"]
    fn size(& self) -> Result < usize >
    {
        if self.order() == 0 { return Ok(1); } let mut size = 1; for dim in
        self.shape() { size *= usize :: try_from(dim) ? ; } Ok(size)
    }
}

```

**Reference feature matrix:** shadowing (item is available without an additional gate)

### Supported usage

```
fn accepts_structure<S: spenso::structure::TensorStructure>(_structure: &S) {}
```

#### Slot

**Kind:** AssociatedType

```
type Slot : IsAbstractSlot + DualSlotTo < Dual = Self :: Slot > ;
```

#### Indexed

**Kind:** AssociatedType

```
type Indexed : TensorStructure < Indexed = Self :: Indexed, Slot = Self ::
Slot > ;
```

#### reindex

**Kind:** Method

```
fn reindex(self, indices : & [< Self :: Slot as IsAbstractSlot > :: Aind],) ->
Result < PermutedStructure < Self :: Indexed > , StructureError > ;
```

#### dual

**Kind:** Method

```
fn dual(self) -> Self;
```

#### string\_rep

**Kind:** Method

```
fn string_rep(& self) -> String
{
    self.external_structure_iter().map(| s | s.to_string()).collect :: < Vec <
    _ > > ().join("")
}

```

#### external\_structure\_iter

**Kind:** Method

```
fn external_structure_iter(& self) -> impl Iterator < Item = Self :: Slot > ;
```

### **external\_dims\_iter**

**Kind:** Method

```
fn external_dims_iter(& self) -> impl Iterator < Item = Dimension > ;
```

### **external\_reps\_iter**

**Kind:** Method

```
fn external_reps_iter(& self,) -> impl Iterator < Item = Representation < < Self :: Slot as IsAbstractSlot > :: R > > ;
```

### **external\_indices\_iter**

**Kind:** Method

```
fn external_indices_iter(& self) -> impl Iterator < Item = < Self :: Slot as IsAbstractSlot > :: Aind > ;
```

### **get\_aind**

**Kind:** Method

```
fn get_aind(& self, i : impl Into < SlotIndex >) -> Option < < Self :: Slot as IsAbstractSlot > :: Aind > ;
```

### **get\_rep**

**Kind:** Method

```
fn get_rep(& self, i : impl Into < SlotIndex > ,) -> Option < Representation < < Self :: Slot as IsAbstractSlot > :: R > > ;
```

### **get\_dim**

**Kind:** Method

```
fn get_dim(& self, i : impl Into < SlotIndex >) -> Option < Dimension > ;
```

### **get\_slot**

**Kind:** Method

```
fn get_slot(& self, i : impl Into < SlotIndex >) -> Option < Self :: Slot > ;
```

### **order**

**Kind:** Method

```
fn order(& self) -> usize;
```

### **external\_structure**

**Kind:** Method

```
#[doc =  
" returns the list of slots that are the external indices of the tensor"] fn  
external_structure(& self) -> Vec < Self :: Slot >  
{ self.external_structure_iter().collect() }  
returns the list of slots that are the external indices of the tensor
```

### **to\_shell**

**Kind:** Method

```
fn to_shell(self) -> TensorShell < Self > where Self : Sized,  
{ TensorShell :: new(self) }
```



### **contains\_matching**

**Kind:** Method

```
fn contains_matching(& self, slot : & Self :: Slot) -> bool
{ self.external_structure_iter().any(| s | s.matches(slot)) }
```

### **external\_reps**

**Kind:** Method

```
fn external_reps(& self) -> Vec < Representation < < Self :: Slot as
IsAbstractSlot > :: R > > { self.external_reps_iter().collect() }
```

### **external\_indices**

**Kind:** Method

```
fn external_indices(& self) -> Vec < < Self :: Slot as IsAbstractSlot > ::
Aind > { self.external_indices_iter().collect() }
```

### **same\_content**

**Kind:** Method

```
#[doc =
" checks if the tensor has the same exact structure as another tensor"] fn
same_content(& self, other : & Self) -> bool { self.same_external(other) }
checks if the tensor has the same exact structure as another tensor
```

### **match\_index**

**Kind:** Method

```
#[doc =
" Given two [`TensorStructure`]s, returns the index of the first matching slot in
each external index list, along with a boolean indicating if there is a single
match"]
fn match_index(& self, other : & Self) -> Option < (bool, usize, usize) >
{
    let posmap =
self.external_structure_iter().enumerate().map(| (i, slot) |
(slot, i)).collect :: < AHashMap < _, _ > > (); let mut first_pair :
Option < (usize, usize) > = None; for (j, slot) in
other.external_structure_iter().enumerate()
{
    if let Some(& i) = posmap.get(& slot.dual())
    {
        if let Some((i, j)) = first_pair { return Some((false, i, j)); }
        first_pair = Some((i, j));
    }
} first_pair.map(| (i, j) | (true, i, j))
}
```

Given two [`TensorStructure`]s, returns the index of the first matching slot in each external index list, along with a boolean indicating if there is a single match

### **match\_indices**

**Kind:** Method

```
#[doc =
" Given two [`TensorStructure`]s, returns the index of the first matching slot in
each external index list"]
```

```

fn match_indices(& self, other : & Self) -> Option <
(Permutation, Vec < bool > , Vec < bool >) >
{
    let mut self_matches = vec! [false; self.order()]; let mut perm = Vec ::
new(); let mut other_matches = vec! [false; other.order()]; let posmap =
self.external_structure_iter().enumerate().map(| (i, slot) |
(slot, i)).collect :: < AHashMap < _, _ > > (); for (j, slot_other) in
other.external_structure_iter().enumerate()
{
    if let Some(& i) = posmap.get(& slot_other.dual())
    { self_matches [i] = true; other_matches [j] = true; perm.push(i); }
} if perm.is_empty() { None } else
{
    let p : Permutation = Permutation :: sort(& perm);
    Some((p, self_matches, other_matches))
}
}

```

Given two [`TensorStructure`]'s, returns the index of the first matching slot in each external index list

### traces

**Kind:** Method

```

#[doc = " Identify the repeated slots in the external index list"] fn
traces(& self) -> Vec < [usize; 2] >
{
    let mut positions : HashMap < < Self as TensorStructure > :: Slot, Vec <
usize > > = HashMap :: new(); for (index, key) in
self.external_structure_iter().enumerate()
{
    if let Some(v) = positions.get_mut(& key.dual()) { v.push(index); }
    else { positions.insert(key, vec! [index]); }
}
positions.into_values().filter_map(| indices |
{
    if indices.len() == 2 { Some([indices [0], indices [1]]) } else
    { None }
}).collect()
}

```

Identify the repeated slots in the external index list

### shape

**Kind:** Method

```

#[doc =
" yields the (outwards facing) shape of the tensor as a list of dimensions"]
fn shape(& self) -> Vec < Dimension > { self.external_dims_iter().collect() }
yields the (outwards facing) shape of the tensor as a list of dimensions

```

### reps

**Kind:** Method

```

fn reps(& self) -> Vec < Representation < < Self :: Slot as IsAbstractSlot >
:: R > > { self.external_reps_iter().collect() }

```

### **same\_external**

**Kind:** Method

```
#[doc = " checks if externally, the two tensors are the same"] fn
same_external(& self, other : & Self) -> bool
{
    let set1 : HashSet < _ > = self.external_structure_iter().collect(); let
    set2 : HashSet < _ > = other.external_structure_iter().collect(); set1 ==
    set2
}
```

checks if externally, the two tensors are the same

### **find\_permutation**

**Kind:** Method

```
#[doc =
" find the permutation of the external indices that would make the two tensors the
same. Applying the permutation to other should make it the same as self"]
fn find_permutation(& self, other : & Self) -> Result < Permutation >
{
    if self.order() != other.order()
    {
        return
        Err(eyre!
            ("Mismatched order: {} vs {}", self.order(), other.order()));
    } let other_structure = other.external_structure(); let self_structure =
    self.external_structure(); let other_sort = Permutation ::
    sort(& other_structure); let self_sort = Permutation ::
    sort(& self_structure); if other_sort.apply_slice(& other_structure) ==
    self_sort.apply_slice(& self_structure)
    { Ok(other_sort.compose(& self_sort.inverse())) } else
    { Err(eyre! ("Mismatched structure")) }
}
```

find the permutation of the external indices that would make the two tensors the same. Applying the permutation to other should make it the same as self

### **strides\_column\_major**

**Kind:** Method

```
#[doc = " yields the strides of the tensor in column major order"] fn
strides_column_major(& self) -> Result < Vec < usize > >
{
    let mut strides : Vec < usize > = vec! [1; self.order()]; if self.order()
    == 0 { return Ok(strides); } for i in 0 .. self.order() - 1
    {
        strides [i + 1] = strides [i] * usize :: try_from(self.shape() [i]) ?
        ;
    } Ok(strides)
}
```

yields the strides of the tensor in column major order

### **strides\_row\_major**

**Kind:** Method

```
#[doc = " yields the strides of the tensor in row major order"] fn
strides_row_major(& self) -> Result < Vec < usize > >
```

```

{
    let mut strides = vec! [1; self.order()]; if self.order() == 0
    { return Ok(strides); } for i in (0 .. self.order() - 1).rev()
    {
        strides [i] = strides [i + 1] * usize ::
            try_from(self.shape() [i + 1]) ? ;
    } Ok(strides)
}

```

yields the strides of the tensor in row major order

### **strides**

**Kind:** Method

```

#[doc = " By default, the strides are row major"] fn strides(& self) -> Result
< Vec < usize > > { self.strides_row_major() }

```

By default, the strides are row major

### **verify\_indices**

**Kind:** Method

```

#[doc =
" Verifies that the list of indices provided are valid for the tensor"
#[doc = "" ] #[doc = " # Errors" ] #[doc = "" ]
#[doc =
" `Mismatched order` = if the length of the indices is different from the order of
the tensor," ]
#[doc = "" ]
#[doc =
" `Index out of bounds` = if the index is out of bounds for the dimension of that
index" ]
#[doc = "" ] fn verify_indices < C : AsRef < [ConcreteIndex] > >
(& self, indices : C) -> Result < () >
{
    if indices.as_ref().len() != self.order()
    {
        return
        Err(eyre!
            ("Mismatched order: {} indices, vs order {}", indices.as_ref().len(),
            self.order()));
    } for (i, dim_len) in
self.external_structure_iter().map(| slot | slot.dim()).enumerate()
{
    if indices.as_ref() [i] >= usize :: try_from(dim_len) ?
    {
        return
        Err(eyre!
            ("Index {} out of bounds for dimension {} of size {}",
            indices.as_ref()[i], i, usize::try_from(dim_len)?));
    }
}
} Ok(())
}

```

Verifies that the list of indices provided are valid for the tensor

**# Errors**

**`Mismatched order`** = if the length of the indices is different from the order of the

tensor,

`Index out of bounds` = if the index is out of bounds for the dimension of that index

### **flat\_index**

**Kind:** Method

```
#[doc = " yields the flat index of the tensor given a list of indices"]
#[doc = ""] #[doc = " # Errors"] #[doc = ""]
#[doc = " Same as [`Self::verify_indices`]"] fn flat_index < C : AsRef <
[ConcreteIndex] > > (& self, indices : C) -> Result < FlatIndex >
{
    let strides = self.strides() ? ; self.verify_indices(& indices) ? ; let
    mut idx = 0; for (i, & index) in indices.as_ref().iter().enumerate()
    { idx += index * strides [i]; } Ok(idx.into())
}
```

yields the flat index of the tensor given a list of indices

# Errors

Same as [`Self::verify\_indices`]

### **expanded\_index**

**Kind:** Method

```
#[doc = " yields the expanded index of the tensor given a flat index"]
#[doc = ""] #[doc = " # Errors"] #[doc = ""]
#[doc =
" `Index out of bounds` = if the flat index is out of bounds for the tensor"]
fn expanded_index(& self, flat_index : FlatIndex) -> Result < ExpandedIndex >
{
    let mut indices = vec! []; let mut index : usize = flat_index.into(); for
    & stride in & self.strides() ?
    { indices.push(index / stride); index %= stride; } if usize ::
    from(flat_index) < self.size() ? { Ok(indices.into()) } else
    { Err(eyre! ("Index {flat_index} out of bounds")) }
}
```

yields the expanded index of the tensor given a flat index

# Errors

`Index out of bounds` = if the flat index is out of bounds for the tensor

### **co\_expanded\_index**

**Kind:** Method

```
fn co_expanded_index(& self, flat_index : FlatIndex) -> Result <
DualConciousExpandedIndex >
{
    let mut indices = vec! []; for (r, i) in
    self.external_reps_iter().zip(self.expanded_index(flat_index) ? .iter())
    {
        if r.rep.is_base() && r.rep.is_dual()
        { indices.push(DualConciousIndex :: SelfDual(* i)); } else if
        r.rep.is_base() { indices.push(DualConciousIndex :: Up(* i)); } else
        { indices.push(DualConciousIndex :: Down(* i)); }
    }
}
```

```
    } Ok(indices.into())
}
```

### **index\_iter**

**Kind:** Method

```
#[doc = " yields an iterator over the indices of the tensor"] fn
index_iter(& self) -> TensorStructureIndexIterator < '_, Self > where Self :
Sized, { TensorStructureIndexIterator :: new(self) }

yields an iterator over the indices of the tensor
```

### **is\_scalar**

**Kind:** Method

```
#[doc = " if the tensor has no (external) indices, it is a scalar"] fn
is_scalar(& self) -> bool { self.order() == 0 }

if the tensor has no (external) indices, it is a scalar
```

### **is\_fully\_self\_dual**

**Kind:** Method

```
fn is_fully_self_dual(& self) -> bool;
```

### **size**

**Kind:** Method

```
#[doc =
" yields the size of the tensor, i.e. the product of the dimensions. This is the
length of the vector of the data in a dense tensor"]
fn size(& self) -> Result < usize >
{
    if self.order() == 0 { return Ok(1); } let mut size = 1; for dim in
    self.shape() { size *= usize :: try_from(dim) ? ; } Ok(size)
}
```

yields the size of the tensor, i.e. the product of the dimensions. This is the length of the vector of the data in a dense tensor

[Exhaustive reference](#) · [Source](#)

### **Dense tensor**

Stores every tensor component in structure-defined flat order.

Stores every tensor component in structure-defined flat order.

```
#[derive(Debug, Clone, PartialEq, Deserialize, Serialize, Hash, Eq, Encode,
Decode)] pub struct DenseTensor < T, S = OrderedStructure >
{ pub data : Vec < T > , pub structure : S, }
```

**Reference feature matrix:** shadowing (item is available without an additional gate)

### **Supported usage**

```
use spenso::tensors::data::DenseTensor;
```

### **data**

**Kind:** Field

Vec < T >

## structure

**Kind:** Field

S

[Exhaustive reference](#) · [Source](#)

## Sparse tensor

Stores only explicit tensor components keyed by flat index.

Sparse data tensor, generic on storage type `T`, and structure type `I`.

Stores data in a hashmap of usize, using ahash's hashmap.

The usize key is the flattened index of the corresponding position in the dense tensor

```
#[doc =  
" Sparse data tensor, generic on storage type `T`, and structure type `I`."]  
#[doc = ""]  
#[doc = " Stores data in a hashmap of usize, using ahash's hashmap."]  
#[doc =  
" The usize key is the flattened index of the corresponding position in the dense  
tensor"]  
#[derive(Debug, Clone, Serialize, Deserialize, PartialEq, Eq, Encode, Decode)]  
pub struct SparseTensor < T, I = OrderedStructure >  
{  
    pub elements : std :: collections :: HashMap < FlatIndex, T > , pub zero :  
        T, pub structure : I,  
}
```

**Reference feature matrix:** shadowing (item is available without an additional gate)

## Supported usage

```
use spenso::tensors::data::SparseTensor;
```

## elements

**Kind:** Field

std :: collections :: HashMap < FlatIndex, T >

## zero

**Kind:** Field

T

## structure

**Kind:** Field

I

[Exhaustive reference](#) · [Source](#)

## Pairwise contraction

Defines pairwise tensor contraction with an explicit settings type and output.

Defines pairwise tensor contraction with an explicit settings type and output.

```
pub trait Contract < T = Self, Settings = () >  
{  
    type LCM; fn contract(& self, other : & T) -> Result < Self :: LCM,
```

```

    ContractionError > ;
}

```

**Reference feature matrix:** shadowing (item is available without an additional gate)

### Supported usage

```
fn can_contract<T: spenso::contraction::Contract>(_tensor: &T) {}
```

### LCM

**Kind:** AssociatedType

```
type LCM;
```

### contract

**Kind:** Method

```
fn contract(& self, other : & T) -> Result < Self :: LCM, ContractionError > ;
```

Exhaustive reference · Source

### Tensor network

Binds symbolic tensor expressions to libraries and an executable contraction graph.

Binds symbolic tensor expressions to libraries and an executable contraction graph.

```
#[derive(Debug, Clone, Serialize, Deserialize, bincode_trait_derive::Encode,
bincode_trait_derive::Decode, PartialEq, Eq,)]
#[cfg_attr(feature = "shadowing",
trait_decode(trait = symbolica::state::HasStateMap),)] pub struct Network < S,
LibKey, FunKey, Aind = AbstractIndex >
{
    pub graph : NetworkGraph < LibKey, FunKey, Aind > , pub store : S, pub
state : NetworkState,
}
```

**Reference feature matrix:** shadowing (item is available without an additional gate)

### Supported usage

```
use spenso::network::Network;
```

### graph

**Kind:** Field

```
NetworkGraph < LibKey, FunKey, Aind >
```

### store

**Kind:** Field

```
S
```

### state

**Kind:** Field

```
NetworkState
```

Exhaustive reference · Source

### Parametric tensor

Stores tensor components as parameter-dependent symbolic coefficients.

Stores tensor components as parameter-dependent symbolic coefficients.



```
#[derive(Clone, Debug, PartialEq, Eq, Hash, bincode_trait_derive::Encode,
bincode_trait_derive::Decode,)]
#[trait_decode(trait = symbolica::state::HasStateMap)] pub struct ParamTensor
< S = OrderedStructure >
{ pub tensor : DataTensor < Atom, S > , pub param_type : ParamOrComposite, }
```

**Requires features:** shadowing

### Supported usage

```
use spenso::tensors::parametric::ParamTensor;
```

#### tensor

**Kind:** Field

DataTensor < Atom, S >

#### param\_type

**Kind:** Field

ParamOrComposite

[Exhaustive reference](#) · [Source](#)

### spenso-macros

Spenso's exported procedural macros. Expansion details remain in the exhaustive Rustdoc sidecar.

#### SimpleRepresentation derive

Derives representation naming and duality boilerplate from the representation helper attribute.

Derives representation naming and duality boilerplate from the representation helper attribute.

```
#[derive(SimpleRepresentation)]
fn derive_simple_representation(input : TokenStream) -> TokenStream
```

**Reference feature matrix:** shadowing (item is available without an additional gate)

### Supported usage

```
#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash, PartialOrd, Ord, Default,
spenso_macros::SimpleRepresentation)] #[representation(name = "flavor", self_dual)]
struct Flavor;
```

[Exhaustive reference](#) · [Source](#)

### spenso-hep-lib

Spenso's high-energy-physics companion library. These entries *complement* the tensor concepts owned by the main Spenso documentation.

#### HEP tensor library

Constructs the standard high-energy-physics tensor library for a chosen scalar data type.

Constructs the standard high-energy-physics tensor library for a chosen scalar data type.

```
fn hep_lib < Aind : AbsInd, T : TensorLibraryData + Clone + Default >
(one : T, zero : T,) -> TensorLibrary < MixedTensor < T, ExplicitKey < Aind >
> , Aind >
```

| Parameter | Type |
|-----------|------|
| one       | T    |
| zero      | T    |

**Returns:** TensorLibrary < MixedTensor < T, ExplicitKey < Aind > > , Aind >

### Supported usage

```
use spenso_hep_lib::hep_lib;
```

Exhaustive reference · Source

### Dirac gamma data

Creates sparse Dirac-basis gamma-matrix data for the supplied tensor structure.

Creates sparse Dirac-basis gamma-matrix data for the supplied tensor structure.

```
fn gamma_data_dirac < T, N > (structure : N, one : T, zero : T) ->
SparseTensor < Complex < T > , N > where T : Clone + Neg < Output = T > , N :
TensorStructure,
```

| Parameter | Type |
|-----------|------|
| structure | N    |
| one       | T    |
| zero      | T    |

**Returns:** SparseTensor < Complex < T > , N >

### Supported usage

```
use spenso_hep_lib::gamma_data_dirac;
```

Exhaustive reference · Source

### SU(3) generator data

Creates the standard sparse SU(3) fundamental-generator tensor.

Fundamental SU(3) generators in the normalization  $\text{Tr}(T^a T^b) = 1/2 \delta^{ab}$ .

The index order follows `CS.t_strct`: adjoint, fundamental, anti-fundamental.

```
fn su3_generator_data < N > (structure : N) -> SparseTensor < Complex < f64 >
, N > where N : TensorStructure,
```

| Parameter | Type |
|-----------|------|
| structure | N    |

**Returns:** SparseTensor < Complex < f64 > , N >

### Supported usage

```
use spenso_hep_lib::su3_generator_data;
```

Exhaustive reference · Source

## spynso3

Exports registered by spynso3.

## Representation

A representation in the sense of group representation theory for tensor indices.

A representation in the sense of group representation theory for tensor indices.

Representations define the transformation properties of tensor indices under group operations.

They specify the dimension and duality structure, determining which indices can contract.

Key concepts:

- **Self-dual**: Indices can contract with other indices of the same representation
- **Dualizable**: Indices can only contract with their dual representation
- **Dimension**: Size of the representation space

Predefined representations are available as class methods:

- `Representation.euc(d)`: Euclidean space (self-dual)
- `Representation.mink(d)`: Minkowski space (self-dual)
- `Representation.bis(d)`: Bispinor (self-dual)
- `Representation.cof(d)`: Color fundamental (dualizable)
- `Representation.coad(d)`: Color adjoint (self-dual)
- `Representation.cos(d)`: Color sextet (dualizable)

# Examples:

```
```python
from symbolica.community.spenso import Representation

# Standard representations
euclidean = Representation.euc(4)      # 4D Euclidean
lorentz = Representation.mink(4)      # 4D Minkowski
color = Representation.cof(3)         # SU(3) fundamental
adjoint = Representation.coad(8)      # SU(3) adjoint

# Custom representation
custom = Representation("MyRep", 5, is_self_dual=True)

# Create slots with indices
mu_slot = euclidean('mu')            # Euclidean index  $\mu$ 
a_slot = color('a')                  # Color index  $a$ 

# Generate metric tensors
metric = euclidean.g('mu', 'nu')     #  $g_{\mu\nu}$ 
```

class Representation:
```

**Requires features:** python\_stubgen

**Introspection scaffold (not a task example)**

```
from symbolica.community.spenso import Representation
help(Representation)
```

`__call__`

**Kind:** Method

```
def __call__(self, aind: builtins.int | Expression | str) -> Slot:
```

Create a slot from this representation, by specifying an index.

Parameters

-----

a<sub>ind</sub> : int, str, or Symbol

The index specification

Returns

-----

Slot

A new Slot object with the specified index

Examples

-----

```
>>> from symbolica.community.spenso import Representation
>>> import symbolica as sp
>>> rep = Representation.euc(3)
>>> slot1 = rep('mu')
>>> slot2 = rep(1)
>>> slot3 = rep(sp.S('nu'))
```

**a<sub>ind</sub>**

**Kind:** Parameter

a<sub>ind</sub>: builtins.int | Expression | str

The index specification

**\_\_call\_\_**

**Kind:** Method

def \_\_call\_\_(self, a<sub>ind</sub>: Expression) -> Expression | Slot:

Create a slot or symbolic expression from this representation.

Parameters

-----

a<sub>ind</sub> : Expression

The index specification (Expression creates symbolic representation)

Returns

-----

Expression

A symbolic expression representing this representation

Examples

-----

```
>>> from symbolica.community.spenso import Representation
>>> import symbolica as sp
>>> rep = Representation.euc(3)
>>> expr = rep(sp.E("cos(x)"))
```

**a<sub>ind</sub>**

**Kind:** Parameter

a<sub>ind</sub>: Expression

The index specification (Expression creates symbolic representation)

## **\_\_new\_\_**

**Kind:** AssociatedFunction

```
def __new__(cls, name: builtins.str, dimension: builtins.int | Expression | str,
is_self_dual: builtins.bool = ...) -> Representation:
```

Create and register a new representation with specified properties.

# Parameters:

- name: String name for the representation
- dimension: Size of the representation (int or symbolic)
- is\_self\_dual: If True, creates self-dual representation; if False, creates dualizable pair

# Examples:

```
```python
from symbolica.community.spenso import Representation
import symbolica as sp

# Self-dual representation (indices contract with themselves)
euclidean = Representation("Euclidean", 4, is_self_dual=True)

# Dualizable representation (needs dual partner for contraction)
vector_up = Representation("VectorUp", 4, is_self_dual=False)

# Symbolic dimension
n = sp.S('n')
general = Representation("General", n, is_self_dual=True)
```
```

## **name**

**Kind:** Parameter

name: builtins.str

## **dimension**

**Kind:** Parameter

dimension: builtins.int | Expression | str

## **is\_self\_dual**

**Kind:** Parameter

is\_self\_dual: builtins.bool = ...

**Default:** ...

## **dual**

**Kind:** Method

```
def dual(self) -> Representation:
```

## **g**

**Kind:** Method

```
def g(self, i: builtins.int | Expression | str, j: builtins.int | Expression | str) -
> TensorIndices:
```

Create a metric tensor for this representation.

```

# Parameters:
- i: First index
- j: Second index

# Examples:
```python
rep = Representation.mink(4)
metric = rep.g('mu', 'nu') # Minkowski metric  $g_{\mu\nu}$ 
```

```

**i**

**Kind:** Parameter

i: builtins.int | Expression | str

**j**

**Kind:** Parameter

j: builtins.int | Expression | str

**flat**

**Kind:** Method

```

def flat(self, i: builtins.int | Expression | str, j: builtins.int | Expression | str) -> TensorIndices:

```

Create a musical isomorphism tensor for this representation.

```

# Parameters:
- i: First index
- j: Second index

```

```

# Examples:
```python
rep = Representation.mink(4)
flat = rep.flat('mu', 'nu') # Flat isomorphism  $b_{\mu\nu}$ 
```

```

**i**

**Kind:** Parameter

i: builtins.int | Expression | str

**j**

**Kind:** Parameter

j: builtins.int | Expression | str

**id**

**Kind:** Method

```

def id(self, i: builtins.int | Expression | str, j: builtins.int | Expression | str) -> TensorIndices:

```

Create an identity tensor for this representation.

```

# Parameters:
- i: First index
- j: Second index

```

```
# Examples:
```python
rep = Representation.cof(3)
identity = rep.id('a', 'b') # Color identity  $\delta_{ab}$ 
```
```

**i**

**Kind:** Parameter

i: builtins.int | Expression | str

**j**

**Kind:** Parameter

j: builtins.int | Expression | str

**\_\_repr\_\_**

**Kind:** Method

def \_\_repr\_\_(self) -> builtins.str:

**\_\_str\_\_**

**Kind:** Method

def \_\_str\_\_(self) -> builtins.str:

**to\_expression**

**Kind:** Method

def to\_expression(self) -> Expression:

Convert the representation to a symbolic expression.

**bis**

**Kind:** AssociatedFunction

def bis(dimension: builtins.int | Expression | str) -> Representation:

Create a bispinor representation.

# Parameters:

- dimension: The dimension of the bispinor space

**dimension**

**Kind:** Parameter

dimension: builtins.int | Expression | str

**euc**

**Kind:** AssociatedFunction

def euc(dimension: builtins.int | Expression | str) -> Representation:

Create a Euclidean space representation.

# Parameters:

- dimension: The dimension of the Euclidean space

**dimension**

**Kind:** Parameter

dimension: builtins.int | Expression | str

**mink**

**Kind:** AssociatedFunction

def mink(dimension: builtins.int | Expression | str) -> Representation:

Create a Minkowski space representation.

# Parameters:

- dimension: The dimension of the Minkowski space

**dimension**

**Kind:** Parameter

dimension: builtins.int | Expression | str

**cof**

**Kind:** AssociatedFunction

def cof(dimension: builtins.int | Expression | str) -> Representation:

Create a color fundamental representation.

# Parameters:

- dimension: The dimension of the color group (e.g., 3 for SU(3))

**dimension**

**Kind:** Parameter

dimension: builtins.int | Expression | str

**coad**

**Kind:** AssociatedFunction

def coad(dimension: builtins.int | Expression | str) -> Representation:

Create a color adjoint representation.

# Parameters:

- dimension: The dimension of the adjoint representation (e.g., 8 for SU(3))

**dimension**

**Kind:** Parameter

dimension: builtins.int | Expression | str

**cos**

**Kind:** AssociatedFunction

def cos(dimension: builtins.int | Expression | str) -> Representation:

Create a color sextet representation.

# Parameters:

- dimension: The dimension of the sextet representation (e.g., 6 for SU(3))

**dimension**

**Kind:** Parameter

dimension: builtins.int | Expression | str



Exhaustive reference · Source

## Slot

A tensor index slot combining a representation with an abstract index.

A tensor index slot combining a representation with an abstract index.

Slots are the building blocks for tensor structures, pairing a representation (which defines transformation properties) with an abstract index identifier. Slots with matching representations and indices can be contracted.

```
# Examples:
```python
from symbolica.community.spenso import Slot, Representation
import symbolica as sp

# Create representation and slots
rep = Representation.euc(3)
slot1 = rep('mu')          # Slot with string index
slot2 = rep(1)              # Slot with integer index
slot3 = rep(sp.S('nu'))    # Slot with symbolic index

# Create custom slot
custom_slot = Slot("MyRep", 4, 'alpha', dual=False)

# Use in tensor structures
from symbolica.community.spenso import TensorIndices
tensor_structure = TensorIndices(slot1, slot2)
```

class Slot:
```

**Requires features:** python\_stubgen

## Introspection scaffold (not a task example)

```
from symbolica.community.spenso import Slot
help(Slot)
```

**\_\_repr\_\_**

**Kind:** Method

```
def __repr__(self) -> builtins.str:
```

**\_\_str\_\_**

**Kind:** Method

```
def __str__(self) -> builtins.str:
```

**dual**

**Kind:** Method

```
def dual(self) -> Slot:
```

**\_\_new\_\_**

**Kind:** AssociatedFunction

```
def __new__(cls, name: builtins.str, dimension: builtins.int, aind: builtins.int |
Expression | str, dual: builtins.bool = ...) -> Slot:
```

Create a new slot with a custom representation and index.

# Parameters:

- name: String name for the representation
- dimension: Size of the representation space
- aind: The abstract index (int, str, or Symbol)
- dual: If True, creates dualizable representation; if False, self-dual

# Examples:

```
```python
from symbolica.community.spenso import Slot
import symbolica as sp

# Self-dual slot
euclidean_slot = Slot("Euclidean", 4, 'mu', dual=False)

# Dualizable slot
vector_slot = Slot("Vector", 4, 'nu', dual=True)

# With symbolic index
sym_index = sp.S('alpha')
symbolic_slot = Slot("Custom", 3, sym_index, dual=False)
```
```

**name**

**Kind:** Parameter

name: builtins.str

**dimension**

**Kind:** Parameter

dimension: builtins.int

**aind**

**Kind:** Parameter

aind: builtins.int | Expression | str

**dual**

**Kind:** Parameter

dual: builtins.bool = ...

**Default:** ...

**to\_expression**

**Kind:** Method

def to\_expression(self) -> Expression:

Convert the slot to a symbolic expression.

# Examples:

```
```python
rep = Representation.euc(3)
slot = rep('mu')
expr = slot.to_expression() # Symbolic representation of the slot
```
```

Exhaustive reference · Source

## Tensor

A tensor class that can be either dense or sparse with flexible data types.

A tensor class that can be either dense or sparse with flexible data types.

The tensor can store data as floats, complex numbers, or symbolic expressions.

Tensors have an associated structure that defines their shape and index properties.

## Examples

```
-----
>>> from symbolica.community.spenso import Tensor, TensorIndices, Representation
>>> structure = TensorIndices(Representation.euc(4)(1))
>>> data = [1.0, 2.0, 3.0, 4.0]
>>> tensor = Tensor.dense(structure, data)
>>> sparse_tensor = Tensor.sparse(structure, float)
```

```
class Tensor:
```

**Requires features:** python\_stubgen

## Introspection scaffold (not a task example)

```
from symbolica.community.spenso import Tensor
help(Tensor)
```

## structure

**Kind:** Method

```
def structure(self) -> TensorIndices:
```

## sparse

**Kind:** AssociatedFunction

```
def sparse(structure: TensorIndices | builtins.list[Slot], type_info: type) ->
Tensor:
```

Create a new sparse empty tensor with the given structure and data type.

## Parameters

```
-----
structure : TensorIndices or list of Slots
The tensor structure defining shape and index properties
type_info : type
The data type - either `float` or `Expression` class
```

## Returns

```
-----
Tensor
A new sparse tensor with all elements initially zero
```

## Examples

```
-----
>>> from symbolica.community.spenso import Tensor, TensorIndices, Representation as R
>>> structure = TensorIndices(R.euc(3)(1), R.euc(3)(2))
>>> sparse_float = Tensor.sparse(structure, float)
>>> sparse_sym = Tensor.sparse(structure, symbolica.Expression)
```

## **structure**

**Kind:** Parameter

structure: TensorIndices | builtins.list[Slot]

The tensor structure defining shape and index properties

## **type\_info**

**Kind:** Parameter

type\_info: type

The data type - either `float` or `Expression` class

## **dense**

**Kind:** AssociatedFunction

```
def dense(structure: TensorIndices | builtins.list[Slot], data:
typing.Sequence[Expression] | typing.Sequence[builtins.float] |
typing.Sequence[builtins.complex]) -> Tensor:
```

Create a new dense tensor with the given structure and data.

### Parameters

-----

structure : TensorIndices or list of Slots

The tensor structure defining shape and index properties

data : list of float, complex, or Expression

The tensor data in row-major order

### Returns

-----

Tensor

A new dense tensor with the specified data

### Examples

-----

```
>>> from symbolica import S
>>> from symbolica.community.spenso import Tensor, TensorIndices, Representation as R
>>> structure = TensorIndices(R.euc(2)(1), R.euc(2)(2))
>>> data = [1.0, 2.0, 3.0, 4.0]
>>> tensor = Tensor.dense(structure, data)
>>> x, y = S("x", "y")
>>> sym_data = [x, y, x * y, x + y]
>>> sym_tensor = Tensor.dense(structure, sym_data)
```

## **structure**

**Kind:** Parameter

structure: TensorIndices | builtins.list[Slot]

The tensor structure defining shape and index properties

## **data**

**Kind:** Parameter

data: typing.Sequence[Expression] | typing.Sequence[builtins.float] |
typing.Sequence[builtins.complex]

The tensor data in row-major order

### **one**

**Kind:** AssociatedFunction

```
def one() -> Tensor:
```

Create a scalar tensor with value 1.0.

Returns

-----

Tensor

A scalar tensor containing the value 1.0

Examples

-----

```
>>> from symbolica.community.spenso import Tensor
>>> one = Tensor.one()
```

### **zero**

**Kind:** AssociatedFunction

```
def zero() -> Tensor:
```

Create a scalar tensor with value 0.0.

Returns

-----

Tensor

A scalar tensor containing the value 0.0

Examples

-----

```
>>> from symbolica.community.spenso import Tensor
>>> zero = Tensor.zero()
```

### **to\_dense**

**Kind:** Method

```
def to_dense(self) -> None:
```

Convert this tensor to dense storage format.

Convert this tensor to dense storage format.

Converts sparse tensors to dense format in-place. Dense tensors are unchanged.

This allocates memory for all tensor elements.

Examples

-----

```
>>> from symbolica.community.spenso import Tensor, TensorIndices, Representation as R
>>> structure = TensorIndices(R.euc(4)(2))
>>> tensor = Tensor.sparse(structure, float)
>>> tensor[0] = 1.0
>>> tensor.to_dense()
```

### **to\_sparse**

**Kind:** Method

```
def to_sparse(self) -> None:
```

Convert this tensor to sparse storage format.

Convert this tensor to sparse storage format.

Converts dense tensors to sparse format in-place, only storing non-zero elements.  
This can save memory for tensors with many zero elements.

Examples

-----

```
>>> from symbolica.community.spenso import Tensor, TensorIndices, Representation as R
>>> structure = TensorIndices(R.euc(2)(2), R.euc(2)(1))
>>> data = [1.0, 0.0, 0.0, 2.0]
>>> tensor = Tensor.dense(structure, data)
>>> tensor.to_sparse()
```

**`__repr__`**

**Kind:** Method

```
def __repr__(self) -> builtins.str:
```

**`__str__`**

**Kind:** Method

```
def __str__(self) -> builtins.str:
```

**`__len__`**

**Kind:** Method

```
def __len__(self) -> builtins.int:
```

**`__getitem__`**

**Kind:** Method

```
def __getitem__(self, item: builtins.slice | builtins.int |
builtins.list[builtins.int]) -> typing.Any:
```

**`item`**

**Kind:** Parameter

```
item: builtins.slice | builtins.int | builtins.list[builtins.int]
```

**`__getitem__`**

**Kind:** Method

```
def __getitem__(self, item: builtins.slice) -> builtins.list[Expression |
builtins.complex | float]:
```

Get tensor elements at the specified range of indices.

Parameters

-----

**item** : slice

Slice object defining the range of indices

Returns

-----

list of float, complex, or Expression

The tensor elements at the specified range

**item**

**Kind:** Parameter

item: builtins.slice

Slice object defining the range of indices

**\_\_getitem\_\_**

**Kind:** Method

```
def __getitem__(self, item: typing.Sequence[builtins.int] | builtins.int) ->
Expression | builtins.complex | float:
```

Get tensor element at the specified index or indices.

Parameters

-----

item : int or list of int

Index specification (int for flat index, list of int for coordinates)

Returns

-----

float, complex, or Expression

The tensor element at the specified index

**item**

**Kind:** Parameter

item: typing.Sequence[builtins.int] | builtins.int

Index specification (int for flat index, list of int for coordinates)

**\_\_setitem\_\_**

**Kind:** Method

```
def __setitem__(self, item: typing.Any, value: typing.Any) -> None:
```

Set tensor element(s) at the specified index or indices.

Parameters

-----

item : int or list of int

Index specification (int for flat index, list of int for coordinates)

value : float, complex, or Expression

The value to set

Examples

-----

```
>>> from symbolica.community.spenso import Tensor, TensorIndices, Representation as R
>>> structure = TensorIndices(R.euc(2)(2), R.euc(2)(1))
>>> tensor = Tensor.sparse(structure, float)
>>> tensor[0] = 4.0
>>> tensor[1, 1] = 1.0
```

**item**

**Kind:** Parameter

item: typing.Any

Index specification (int for flat index, list of int for coordinates)

## **value**

**Kind:** Parameter

value: typing.Any

The value to set

## **\_\_setitem\_\_**

**Kind:** Method

```
def __setitem__(self, item: builtins.int | typing.Sequence[builtins.int], value: Expression | builtins.complex | float) -> None:
```

Set tensor element at the specified index.

## Parameters

-----

item : int or list of int

Index specification (int for flat index, list of int for coordinates)

value : float, complex, or Expression

The value to set

## Examples

-----

```
>>> from symbolica.community.spenso import Tensor, TensorIndices, Representation
>>> rep = Representation.euc(2)
>>> structure = TensorIndices(rep(1), rep(2))
>>> tensor = Tensor.sparse(structure, float)
>>> tensor[0] = 1.0
>>> tensor[1, 1] = 2.0
```

## **item**

**Kind:** Parameter

item: builtins.int | typing.Sequence[builtins.int]

Index specification (int for flat index, list of int for coordinates)

## **value**

**Kind:** Parameter

value: Expression | builtins.complex | float

The value to set

## **evaluator**

**Kind:** Method

```
def evaluator(self, constants: typing.Mapping[Expression, Expression], funs: typing.Mapping[tuple[Expression, builtins.str, typing.Sequence[Expression]], Expression], params: typing.Sequence[Expression], iterations: builtins.int = ..., n_cores: builtins.int = ..., verbose: builtins.bool = ...) -> TensorEvaluator:
```

Create an optimized evaluator for symbolic tensor expressions.

Create an optimized evaluator for symbolic tensor expressions.

Compiles the symbolic expressions in this tensor into an optimized evaluation tree that can efficiently compute numerical values for different parameter inputs.

## Parameters



-----

**constants** : dict

Dict mapping symbolic expressions to their constant numerical values

**funcs** : dict

Dict mapping function signatures to their symbolic definitions

**params** : list of Expression

List of symbolic parameters that will be varied during evaluation

**iterations** : int, optional

Number of optimization iterations for Horner scheme (default: 100)

**n\_cores** : int, optional

Number of CPU cores to use for optimization (default: 4)

**verbose** : bool, optional

Whether to print optimization progress (default: False)

Returns

-----

TensorEvaluator

An optimized evaluator for efficient numerical evaluation

Examples

-----

```
>>> from symbolica import S
```

```
>>> from symbolica.community.spenso import Tensor, TensorIndices, Representation as R
```

```
>>> x, y = S("x", "y")
```

```
>>> structure = TensorIndices(R.euc(2)(1))
```

```
>>> tensor = Tensor.dense(structure, [x * y, x + y])
```

```
>>> evaluator = tensor.evaluator(constants={}, funcs={}, params=[x, y], iterations=50)
```

```
>>> results = evaluator.evaluate_complex([[1.0, 2.0], [3.0, 4.0]])
```

**constants**

**Kind:** Parameter

constants: typing.Mapping[Expression, Expression]

Dict mapping symbolic expressions to their constant numerical values

**funcs**

**Kind:** Parameter

funcs: typing.Mapping[tuple[Expression, builtins.str, typing.Sequence[Expression]], Expression]

Dict mapping function signatures to their symbolic definitions

**params**

**Kind:** Parameter

params: typing.Sequence[Expression]

List of symbolic parameters that will be varied during evaluation

**iterations**

**Kind:** Parameter

iterations: builtins.int = ...

**Default:** ...

Number of optimization iterations for Horner scheme (default: 100)

## **n\_cores**

**Kind:** Parameter

n\_cores: builtins.int = ...

**Default:** ...

Number of CPU cores to use for optimization (default: 4)

## **verbose**

**Kind:** Parameter

verbose: builtins.bool = ...

**Default:** ...

Whether to print optimization progress (default: False)

## **scalar**

**Kind:** Method

def scalar(self) -> Expression:

Extract the scalar value from a rank-0 (scalar) tensor.

Returns

-----

Expression

The scalar expression contained in this tensor

Raises

-----

RuntimeError

If the tensor is not a scalar

Examples

-----

```
>>> from symbolica.community.spenso import Tensor
```

```
>>> scalar_tensor = Tensor.one()
```

```
>>> value = scalar_tensor.scalar()
```

## **\_\_iter\_\_**

**Kind:** Method

def \_\_iter\_\_(self) -> typing.Iterator[typing.Any]:

Iterator

Exhaustive reference · Source

## **TensorIndices**

A tensor structure with abstract indices for symbolic tensor operations.

A tensor structure with abstract indices for symbolic tensor operations.

TensorIndices represents the index structure of tensors with named abstract indices that can be contracted, manipulated symbolically, and converted to expressions.

It maintains both the representation structure and index assignments.

Examples

-----

```
>>> from symbolica.community.spenso import TensorIndices, Representation, TensorName
>>> rep = Representation.euc(3)
>>> mu = rep('mu')
>>> nu = rep('nu')
>>> indices = TensorIndices(mu, nu)
>>> T = TensorName("T")
>>> named_indices = T(mu, nu)
>>> expr = named_indices.to_expression()

class TensorIndices:
```

**Requires features:** python\_stubgen

### Introspection scaffold (not a task example)

```
from symbolica.community.spenso import TensorIndices
help(TensorIndices)
```

#### set\_name

**Kind:** Method

```
def set_name(self, name: TensorName | builtins.str | Expression) -> None:
    Set the tensor name for this structure.
```

#### Parameters

-----

name : TensorName

The tensor name to assign

#### Examples

-----

```
>>> from symbolica.community.spenso import TensorIndices, TensorName, Representation
>>> rep = Representation.euc(3)
>>> structure = TensorIndices(rep('mu'), rep('nu'))
>>> T = TensorName("T")
>>> structure.set_name(T)
```

#### name

**Kind:** Parameter

name: TensorName | builtins.str | Expression

The tensor name to assign

#### get\_name

**Kind:** Method

```
def get_name(self) -> typing.Optional[TensorName]:
    Get the tensor name of this structure.
```

#### Returns

-----

TensorName or None

The tensor name if set, None otherwise

#### Examples

-----

```
>>> name = structure.get_name()
```

**\_\_repr\_\_**

**Kind:** Method

```
def __repr__(self) -> builtins.str:
```

**\_\_str\_\_**

**Kind:** Method

```
def __str__(self) -> builtins.str:
```

**to\_expression**

**Kind:** Method

```
def to_expression(self) -> Expression:
```

Convert the tensor indices to a symbolic expression.

Creates a symbolic representation of the tensor with its indices that can be used in algebraic manipulations and pattern matching.

Returns

-----

Expression

A symbolic Expression representing this indexed tensor

Raises

-----

RuntimeError

If the tensor structure has no name

Examples

-----

```
>>> from symbolica.community.spenso import TensorName, Representation
```

```
>>> T = TensorName("T")
```

```
>>> rep = Representation.euc(3)
```

```
>>> mu = rep('mu')
```

```
>>> nu = rep('nu')
```

```
>>> indices = T(mu, nu)
```

```
>>> expr = indices.to_expression()
```

**\_\_len\_\_**

**Kind:** Method

```
def __len__(self) -> builtins.int:
```

**\_\_add\_\_**

**Kind:** Method

```
def __add__(self, rhs: Expression | int | str | float | builtins.complex |  
TensorIndices | Expression) -> Expression:
```

Add this expression to `other`, returning the result.

**rhs**

**Kind:** Parameter

rhs: Expression | int | str | float | builtins.complex | TensorIndices | Expression

**`__radd__`**

**Kind:** Method

```
def __radd__(self, rhs: Expression | int | str | float | builtins.complex |  
TensorIndices | Expression) -> Expression:
```

Add this expression to `other`, returning the result.

**rhs**

**Kind:** Parameter

rhs: Expression | int | str | float | builtins.complex | TensorIndices | Expression

**`__sub__`**

**Kind:** Method

```
def __sub__(self, rhs: Expression | int | str | float | builtins.complex |  
TensorIndices | Expression) -> Expression:
```

Subtract `other` from this expression, returning the result.

**rhs**

**Kind:** Parameter

rhs: Expression | int | str | float | builtins.complex | TensorIndices | Expression

**`__rsub__`**

**Kind:** Method

```
def __rsub__(self, rhs: Expression | int | str | float | builtins.complex |  
TensorIndices | Expression) -> Expression:
```

Subtract this expression from `other`, returning the result.

**rhs**

**Kind:** Parameter

rhs: Expression | int | str | float | builtins.complex | TensorIndices | Expression

**`__mul__`**

**Kind:** Method

```
def __mul__(self, rhs: Expression | int | str | float | builtins.complex |  
TensorIndices | Expression) -> Expression:
```

Add this expression to `other`, returning the result.

**rhs**

**Kind:** Parameter

rhs: Expression | int | str | float | builtins.complex | TensorIndices | Expression

**`__rmul__`**

**Kind:** Method

```
def __rmul__(self, rhs: Expression | int | str | float | builtins.complex |  
TensorIndices | Expression) -> Expression:
```

Add this expression to `other`, returning the result.

**rhs**

**Kind:** Parameter

rhs: Expression | int | str | float | builtins.complex | TensorIndices | Expression

### **\_\_new\_\_**

**Kind:** AssociatedFunction

```
def __new__(cls, *slots: TensorIndices | builtins.list[Slot], name: TensorName |
builtins.str | Expression | None = None) -> TensorIndices:
```

Create tensor structure from slots and optional arguments.

#### Parameters

-----

\*additional\_args : Slot or Expression

Mixed arguments (Slot objects and Expressions for additional arguments)

name : TensorName, optional

Optional tensor name to assign to the structure

#### Returns

-----

TensorIndices

A new TensorIndices object

#### Examples

-----

```
>>> from symbolica import S
```

```
>>> from symbolica.community.spenso import TensorIndices, Representation, TensorName
```

```
>>> rep = Representation.euc(3)
```

```
>>> mu = rep('mu')
```

```
>>> nu = rep('nu')
```

```
>>> structure = TensorIndices(mu, nu)
```

```
>>> x = S('x')
```

```
>>> structure_with_args = TensorIndices(mu, nu, x)
```

```
>>> T = TensorName("T")
```

```
>>> named_structure = TensorIndices(mu, nu, name=T)
```

### **slots**

**Kind:** Parameter

\*slots: TensorIndices | builtins.list[Slot]

### **name**

**Kind:** Parameter

name: TensorName | builtins.str | Expression | None = None

**Default:** None

Optional tensor name to assign to the structure

### **\_\_getitem\_\_**

**Kind:** Method

```
def __getitem__(self, item: builtins.slice) -> builtins.list[Expression |
builtins.complex | float]:
```

Get expanded indices at the specified range of flattened indices.

#### Parameters

-----

item : slice

Slice object defining the range of indices

Returns

-----

list of list of int

List of expanded indices

**item**

**Kind:** Parameter

item: builtins.slice

Slice object defining the range of indices

**\_\_getitem\_\_**

**Kind:** Method

```
def __getitem__(self, item: typing.Sequence[builtins.int]) -> Expression |  
builtins.complex | float:
```

Get flattened index associated to this expanded index.

Parameters

-----

item : list of int

Multi-dimensional index coordinates

Returns

-----

int

The flat index

**item**

**Kind:** Parameter

item: typing.Sequence[builtins.int]

Multi-dimensional index coordinates

**\_\_getitem\_\_**

**Kind:** Method

```
def __getitem__(self, item: builtins.int) -> Expression | builtins.complex | float:
```

Get expanded index associated to this flat index.

Parameters

-----

item : int

Flat index into the tensor

Returns

-----

list of int

Multi-dimensional index coordinates

**item**

**Kind:** Parameter

item: builtins.int

Flat index into the tensor

Exhaustive reference · Source

### TensorLibrary

A library for registering and managing tensor templates and structures.

A library for registering and managing tensor templates and structures.

The TensorLibrary provides a centralized registry for tensor definitions that can be reused across tensor networks and expressions. It manages tensor structures with their associated names and can resolve symbolic references to registered tensors.

```
```python
import symbolica
from symbolica.community.spenso import TensorLibrary, LibraryTensor, TensorStructure,
Representation

lib = TensorLibrary()
rep = Representation.euc(3)
name = symbolica.S("my_tensor")
structure = TensorStructure(rep, rep, name=name)
tensor = LibraryTensor.dense(structure, [1.0, 0.0, 0.0, 1.0, 0.0, 0.0, 1.0, 0.0,
0.0])
lib.register(tensor)
tensor_ref = lib[name]
```
```

```
class TensorLibrary:
```

**Requires features:** python\_stubgen

### Introspection scaffold (not a task example)

```
from symbolica.community.spenso import TensorLibrary
help(TensorLibrary)
```

**\_\_new\_\_**

**Kind:** AssociatedFunction

```
def __new__(cls) -> TensorLibrary:
```

Create a new empty tensor library.

Initializes an empty library ready for registering tensor structures.  
The library automatically manages internal tensor IDs.

Returns

-----

TensorLibrary

A new empty tensor library

Examples

-----

```
>>> from symbolica.community.spenso import TensorLibrary
>>> lib = TensorLibrary()
```

**construct**

**Kind:** AssociatedFunction



```
def construct() -> TensorLibrary:
```

Create a new empty tensor library (static method).

Initializes an empty library ready for registering tensor structures.  
The library automatically manages internal tensor IDs.

Returns

-----

TensorLibrary

A new empty tensor library

Examples

-----

```
>>> from symbolica.community.spenso import TensorLibrary
>>> lib = TensorLibrary.construct()
```

**register**

**Kind:** Method

```
def register(self, tensor: Tensor | LibraryTensor) -> None:
```

Register a tensor in the library.

Adds a tensor template to the library that can be referenced by name  
in tensor networks and symbolic expressions. The tensor must have a name  
set in its structure.

Parameters

-----

tensor : LibraryTensor or Tensor

The tensor to register - can be a LibraryTensor or regular Tensor

Examples

-----

```
>>> import symbolica
>>> from symbolica.community.spenso import TensorLibrary, LibraryTensor,
TensorStructure, Representation
>>> lib = TensorLibrary()
>>> rep = Representation.euc(3)
>>> name = symbolica.S("my_tensor")
>>> structure = TensorStructure(rep, rep, name=name)
>>> tensor = LibraryTensor.dense(structure, [1.0, 0.0, 0.0, 1.0, 0.0, 0.0, 1.0, 0.0,
0.0])
>>> lib.register(tensor)
>>> tensor_ref = lib[name]
```

**tensor**

**Kind:** Parameter

tensor: Tensor | LibraryTensor

The tensor to register - can be a LibraryTensor or regular Tensor

**\_\_getitem\_\_**

**Kind:** Method

```
def __getitem__(self, key: Expression | int | str | float | builtins.complex |
builtins.str) -> TensorStructure:
```

Retrieve a registered tensor structure by name.

Looks up a previously registered tensor by its name and returns a reference structure that can be used to create new tensor instances.

#### Parameters

-----

key : str or Expression

The tensor name - can be a string or symbolic expression

#### Returns

-----

TensorStructure

A TensorStructure representing the registered tensor template

#### Raises

-----

RuntimeError

If the tensor name is not found in the library

#### Examples

-----

```
>>> structure = lib["T"]
```

#### key

**Kind:** Parameter

key: Expression | int | str | float | builtins.complex | builtins.str

The tensor name - can be a string or symbolic expression

#### hep\_lib

**Kind:** AssociatedFunction

```
def hep_lib() -> TensorLibrary:
```

Create a library pre-loaded with High Energy Physics tensor definitions.

Returns a library containing standard HEP tensors such as gamma matrices, color generators, metric tensors, and other commonly used structures in particle physics calculations. They are floating point tensors with f64 precision.

#### Returns

-----

TensorLibrary

A TensorLibrary pre-populated with HEP tensor definitions

#### Examples

-----

```
>>> import symbolica
>>> from symbolica.community.spenso import TensorLibrary, TensorName
>>> hep_lib = TensorLibrary.hep_lib()
>>> gamma_structure = hep_lib[symbolica.S("spenso::gamma")]
```

#### hep\_lib\_atom

**Kind:** AssociatedFunction

```
def hep_lib_atom() -> TensorLibrary:
```

Create a library pre-loaded with High Energy Physics tensor definitions.

Returns a library containing standard HEP tensors such as gamma matrices, color generators, metric tensors, and other commonly used structures in particle physics calculations. They are tensors with atom numeric entries.

Returns

-----

TensorLibrary

A TensorLibrary pre-populated with HEP tensor definitions

Examples

-----

```
>>> import symbolica
>>> from symbolica.community.spenso import TensorLibrary, TensorName
>>> hep_lib = TensorLibrary.hep_lib_atom()
>>> gamma_structure = hep_lib[symbolica.S("spenso::gamma")]
```

Exhaustive reference · Source

### TensorNetwork

A tensor network representing computational graphs of tensor operations.

A tensor network representing computational graphs of tensor operations.

A tensor network is a graph-based representation of tensor computations where nodes represent tensors and operations, edges represent tensor contractions and data flow, and the network can be optimized and executed to compute results.

Tensor networks are particularly useful for symbolic manipulation of complex tensor expressions, optimization of tensor contraction orders, efficient evaluation of large tensor computations, and physics calculations involving many-body systems.

Examples

-----

```
>>> import symbolica as sp
>>> from symbolica.community.spenso import TensorNetwork, Tensor, TensorIndices
>>> x = sp.symbol('x')
>>> expr = x * sp.symbol('T')(sp.symbol('mu'), sp.symbol('nu'))
>>> network = TensorNetwork(expr)
>>> network.execute()
>>> result = network.result_tensor()
```

class TensorNetwork:

**Requires features:** python\_stubgen

### Introspection scaffold (not a task example)

```
from symbolica.community.spenso import TensorNetwork
help(TensorNetwork)
```

\_\_new\_\_

**Kind:** AssociatedFunction

```
def __new__(cls, expr: Expression | int | str | float | builtins.complex |
TensorIndices | Expression, library: typing.Optional[TensorLibrary] = None) ->
TensorNetwork:
```

Create a tensor network by parsing an arithmetic expression.

Parses symbolic expressions containing tensor operations and converts them into an optimizable computational graph representation.

Parameters

-----

**expr** : ArithmeticStructure

The arithmetic expression or tensor structure to parse

**library** : TensorLibrary, optional

Optional tensor library for resolving named tensor references

Returns

-----

TensorNetwork

A new TensorNetwork representing the parsed expression

**expr**

**Kind:** Parameter

expr: Expression | int | str | float | builtins.complex | TensorIndices | Expression

The arithmetic expression or tensor structure to parse

**library**

**Kind:** Parameter

library: typing.Optional[TensorLibrary] = None

**Default:** None

Optional tensor library for resolving named tensor references

**one**

**Kind:** AssociatedFunction

def one() -> TensorNetwork:

Create a tensor network representing the scalar value 1.

Returns

-----

TensorNetwork

A TensorNetwork containing only the scalar 1

Examples

-----

```
>>> from symbolica.community.spenso import TensorNetwork
```

```
>>> one_net = TensorNetwork.one()
```

```
>>> result = one_net.result_scalar()
```

**bracket**

**Kind:** AssociatedFunction

def bracket() -> Expression:

**broadcast**

**Kind:** AssociatedFunction

def broadcast(str: builtins.str) -> Expression:

**str**

**Kind:** Parameter

str: builtins.str

**zero**

**Kind:** AssociatedFunction

def zero() -> TensorNetwork:

Create a tensor network representing the scalar value 0.

Returns

-----

TensorNetwork

A TensorNetwork containing only the scalar 0

Examples

-----

```
>>> from symbolica.community.spenso import TensorNetwork
```

```
>>> zero_net = TensorNetwork.zero()
```

```
>>> result = zero_net.result_scalar()
```

**replace**

**Kind:** Method

```
def replace(self, pattern: Expression | int | str | float | builtins.complex, rhs:
Expression | int | str | float | builtins.complex | HeldExpression |
typing.Callable[[dict[Expression, Expression]], Expression] | int | float | complex |
decimal.Decimal, _cond: typing.Optional[PatternRestriction | Condition] = None,
non_greedy_wildcards: typing.Optional[typing.Sequence[Expression]] = None,
level_range: typing.Optional[tuple[builtins.int, typing.Optional[builtins.int]]] =
None, level_is_tree_depth: typing.Optional[builtins.bool] = None,
allow_new_wildcards_on_rhs: typing.Optional[builtins.bool] = None, rhs_cache_size:
typing.Optional[builtins.int] = None, repeat: typing.Optional[builtins.bool] = None)
-> TensorNetwork:
```

Replace patterns in the tensor network using symbolic pattern matching.

Applies pattern-based transformations to the network structure, allowing for symbolic simplifications, substitutions, and algebraic manipulations.

Parameters

-----

pattern : Expression

The symbolic pattern to match against

rhs : Expression

The replacement expression or pattern

non\_greedy\_wildcards : list of Expression, optional

List of wildcard symbols to match non-greedily

level\_range : tuple of int, optional

Tuple specifying depth range for pattern matching

level\_is\_tree\_depth : bool, optional

Whether level refers to tree depth or expression depth

allow\_new\_wildcards\_on\_rhs : bool, optional

Allow new wildcards in replacement pattern

rhs\_cache\_size : int, optional

Size of cache for replacement pattern compilation

repeat : bool, optional  
Whether to repeatedly apply the replacement until no more matches

Returns

-----

TensorNetwork  
A new TensorNetwork with the replacements applied

#### **pattern**

**Kind:** Parameter

pattern: Expression | int | str | float | builtins.complex

The symbolic pattern to match against

#### **rhs**

**Kind:** Parameter

rhs: Expression | int | str | float | builtins.complex | HeldExpression |  
typing.Callable[[dict[Expression, Expression], Expression] | int | float | complex |  
decimal.Decimal

The replacement expression or pattern

#### **\_cond**

**Kind:** Parameter

\_cond: typing.Optional[PatternRestriction | Condition] = None

**Default:** None

#### **non\_greedy\_wildcards**

**Kind:** Parameter

non\_greedy\_wildcards: typing.Optional[typing.Sequence[Expression]] = None

**Default:** None

List of wildcard symbols to match non-greedily

#### **level\_range**

**Kind:** Parameter

level\_range: typing.Optional[tuple[builtins.int, typing.Optional[builtins.int]]] =  
None

**Default:** None

Tuple specifying depth range for pattern matching

#### **level\_is\_tree\_depth**

**Kind:** Parameter

level\_is\_tree\_depth: typing.Optional[builtins.bool] = None

**Default:** None

Whether level refers to tree depth or expression depth

#### **allow\_new\_wildcards\_on\_rhs**

**Kind:** Parameter

allow\_new\_wildcards\_on\_rhs: typing.Optional[builtins.bool] = None

**Default:** None

Allow new wildcards in replacement pattern

**rhs\_cache\_size**

**Kind:** Parameter

rhs\_cache\_size: typing.Optional[builtins.int] = None

**Default:** None

Size of cache for replacement pattern compilation

**repeat**

**Kind:** Parameter

repeat: typing.Optional[builtins.bool] = None

**Default:** None

Whether to repeatedly apply the replacement until no more matches

**evaluate**

**Kind:** Method

```
def evaluate(self, constants: typing.Mapping[Expression, builtins.float], functions:
typing.Mapping[Expression, typing.Any]) -> TensorNetwork:
```

Evaluate symbolic expressions in the network with numerical values.

Substitutes symbolic constants and functions with numerical values,  
converting symbolic parts of the network to concrete numerical tensors.

Parameters

-----

constants : dict

Dict mapping symbolic expressions to their numerical values

functions : dict

Dict mapping function symbols to Python callable objects

Returns

-----

TensorNetwork

A new TensorNetwork with symbolic expressions evaluated

**constants**

**Kind:** Parameter

constants: typing.Mapping[Expression, builtins.float]

Dict mapping symbolic expressions to their numerical values

**functions**

**Kind:** Parameter

functions: typing.Mapping[Expression, typing.Any]

Dict mapping function symbols to Python callable objects

**execute**

**Kind:** Method

```
def execute(self, library: typing.Optional[TensorLibrary] = None, function_library:
None = None, n_steps: typing.Optional[builtins.int] = None, mode: ExecutionMode
= ...) -> None:
```

Execute the tensor network to perform tensor contractions and simplifications.

Processes the computational graph by executing tensor operations such as contractions, additions, and multiplications. The execution can be controlled by mode and step limits.

#### Parameters

-----

**library** : TensorLibrary, optional

Optional tensor library for resolving tensor operations

**function\_library** : None, optional

Reserved for an internally supplied function library

**n\_steps** : int, optional

Maximum number of execution steps (None for complete execution)

**mode** : ExecutionMode, optional

Execution strategy (ExecutionMode.All, ExecutionMode.Scalar, or ExecutionMode.Single)

#### Examples

-----

```
>>> from symbolica.community.spenso import TensorNetwork, ExecutionMode,
TensorLibrary
```

```
>>> network = TensorNetwork(some_expression)
```

```
>>> network.execute()
```

```
>>> network.execute(n_steps=5)
```

```
>>> network.execute(mode=ExecutionMode.Scalar)
```

```
>>> lib = TensorLibrary.hep_lib()
```

```
>>> network.execute(library=lib)
```

#### **library**

**Kind:** Parameter

library: typing.Optional[TensorLibrary] = None

**Default:** None

Optional tensor library for resolving tensor operations

#### **function\_library**

**Kind:** Parameter

function\_library: None = None

**Default:** None

Reserved for an internally supplied function library

#### **n\_steps**

**Kind:** Parameter

n\_steps: typing.Optional[builtins.int] = None

**Default:** None

Maximum number of execution steps (None for complete execution)



**mode**

**Kind:** Parameter

mode: ExecutionMode = ...

**Default:** ...

Execution strategy (ExecutionMode.All, ExecutionMode.Scalar, or ExecutionMode.Single)

**result\_tensor**

**Kind:** Method

```
def result_tensor(self, library: typing.Optional[TensorLibrary] = None) -> Tensor:
```

Extract the final tensor result from the executed network.

After network execution, retrieves the computed tensor result. The network should be executed before calling this method.

Parameters

-----

library : TensorLibrary, optional

Optional tensor library for resolving tensor structures

Returns

-----

Tensor

The computed tensor result

Raises

-----

RuntimeError

If the network execution resulted in an error

Examples

-----

```
>>> from symbolica.community.spenso import TensorNetwork, TensorLibrary
>>> network = TensorNetwork(tensor_expression)
>>> network.execute()
>>> result = network.result_tensor()
>>> lib = TensorLibrary.hep_lib()
>>> result_with_lib = network.result_tensor(library=lib)
```

**library**

**Kind:** Parameter

library: typing.Optional[TensorLibrary] = None

**Default:** None

Optional tensor library for resolving tensor structures

**result\_scalar**

**Kind:** Method

```
def result_scalar(self) -> Expression:
```

Extract the final scalar result from the executed network.

For networks that evaluate to scalar expressions, retrieves the computed

scalar value. The network should be executed before calling this method.

Returns

-----

Expression

The computed scalar expression

Raises

-----

RuntimeError

If the network execution resulted in an error

Examples

-----

```
>>> from symbolica.community.spenso import TensorNetwork
>>> network = TensorNetwork(scalar_expression)
>>> network.execute()
>>> scalar_result = network.result_scalar()
```

**`__str__`**

**Kind:** Method

```
def __str__(self) -> builtins.str:
```

Return a string representation of the network structure.

Generates a DOT format representation of the computational graph that can be visualized using graphviz or similar tools.

**`__add__`**

**Kind:** Method

```
def __add__(self, rhs: Expression | int | str | float | builtins.complex |
TensorIndices | Expression | TensorNetwork | Tensor) -> TensorNetwork:
```

Add two tensor networks element-wise.

Parameters

-----

rhs : TensorNetwork

The tensor network to add (right-hand side)

Returns

-----

TensorNetwork

A new TensorNetwork representing the sum

Examples

-----

```
>>> net1 = TensorNetwork(expr1)
>>> net2 = TensorNetwork(expr2)
>>> sum_net = net1 + net2
```

**rhs**

**Kind:** Parameter

rhs: Expression | int | str | float | builtins.complex | TensorIndices | Expression | TensorNetwork | Tensor

The tensor network to add (right-hand side)

**`__radd__`**

**Kind:** Method

```
def __radd__(self, rhs: Expression | int | str | float | builtins.complex |
TensorIndices | Expression | TensorNetwork | Tensor) -> TensorNetwork:
```

Add two tensor networks element-wise (right-hand addition).

**`rhs`**

**Kind:** Parameter

`rhs: Expression | int | str | float | builtins.complex | TensorIndices | Expression | TensorNetwork | Tensor`

**`__sub__`**

**Kind:** Method

```
def __sub__(self, rhs: Expression | int | str | float | builtins.complex |
TensorIndices | Expression | TensorNetwork | Tensor) -> TensorNetwork:
```

Subtract one tensor network from another element-wise.

Parameters

-----

`rhs : TensorNetwork`

The tensor network to subtract (right-hand side)

Returns

-----

`TensorNetwork`

A new `TensorNetwork` representing the difference

Examples

-----

```
>>> net1 = TensorNetwork(expr1)
```

```
>>> net2 = TensorNetwork(expr2)
```

```
>>> diff_net = net1 - net2
```

**`rhs`**

**Kind:** Parameter

`rhs: Expression | int | str | float | builtins.complex | TensorIndices | Expression | TensorNetwork | Tensor`

The tensor network to subtract (right-hand side)

**`__rsub__`**

**Kind:** Method

```
def __rsub__(self, rhs: Expression | int | str | float | builtins.complex |
TensorIndices | Expression | TensorNetwork | Tensor) -> TensorNetwork:
```

Subtract one tensor network from another (right-hand subtraction).

**`rhs`**

**Kind:** Parameter

`rhs: Expression | int | str | float | builtins.complex | TensorIndices | Expression | TensorNetwork | Tensor`

**\_\_mul\_\_**

**Kind:** Method

```
def __mul__(self, rhs: Expression | int | str | float | builtins.complex |
TensorIndices | Expression | TensorNetwork | Tensor) -> TensorNetwork:
```

Multiply two tensor networks.

Parameters

-----

rhs : TensorNetwork

The tensor network to multiply with (right-hand side)

Returns

-----

TensorNetwork

A new TensorNetwork representing the product

Examples

-----

```
>>> net1 = TensorNetwork(expr1)
```

```
>>> net2 = TensorNetwork(expr2)
```

```
>>> product_net = net1 * net2
```

**rhs**

**Kind:** Parameter

rhs: Expression | int | str | float | builtins.complex | TensorIndices | Expression | TensorNetwork | Tensor

The tensor network to multiply with (right-hand side)

**\_\_rmul\_\_**

**Kind:** Method

```
def __rmul__(self, rhs: Expression | int | str | float | builtins.complex |
TensorIndices | Expression | TensorNetwork | Tensor) -> TensorNetwork:
```

Multiply two tensor networks (right-hand multiplication).

**rhs**

**Kind:** Parameter

rhs: Expression | int | str | float | builtins.complex | TensorIndices | Expression | TensorNetwork | Tensor

[Exhaustive reference](#) · [Source](#)

## TensorStructure

A tensor structure without abstract indices, defined purely by representations.

A tensor structure without abstract indices, defined purely by representations.

TensorStructure represents the shape and representation structure of tensors without specific index assignments. It's used for defining tensor templates in libraries and for creating indexless tensor computations.

# Examples:

```
```python
```

```
from symbolica.community.spenso import TensorStructure, Representation, TensorName
```

```

# Create from representations
rep = Representation.euc(3)
structure = TensorStructure(rep, rep) # 3x3 matrix structure

# With name for library registration
T = TensorName("T")
named_structure = TensorStructure(rep, rep, name=T)

# Use to create indexed tensor
indices = structure.index('mu', 'nu') # Assign specific indices

# Create symbolic expression
expr = structure.symbolic('a', 'b') # T(a, b)
` ``

```

```
class TensorStructure:
```

**Requires features:** python\_stubgen

### Introspection scaffold (not a task example)

```

from symbolica.community.spenso import TensorStructure
help(TensorStructure)

```

**set\_name**

**Kind:** Method

```
def set_name(self, name: TensorName | builtins.str | Expression) -> None:
```

**name**

**Kind:** Parameter

```
name: TensorName | builtins.str | Expression
```

**get\_name**

**Kind:** Method

```
def get_name(self) -> typing.Optional[TensorName]:
```

**\_\_repr\_\_**

**Kind:** Method

```
def __repr__(self) -> builtins.str:
```

**\_\_str\_\_**

**Kind:** Method

```
def __str__(self) -> builtins.str:
```

**\_\_len\_\_**

**Kind:** Method

```
def __len__(self) -> builtins.int:
```

**\_\_new\_\_**

**Kind:** AssociatedFunction

```

def __new__(cls, *reps_and_additional_args: TensorIndices | builtins.list[Slot],
name: TensorName | builtins.str | Expression | None = None) -> TensorStructure:

```

Construct a new TensorStructure with the given representations.

Parameters

-----

\*reps\_and\_additional\_args : Representation or Expression

Mixed arguments (Representation objects and Expressions for additional arguments)

name : TensorName, optional

Optional tensor name to assign to the structure

Returns

-----

TensorStructure

A new TensorStructure object

Examples

-----

```
>>> from symbolica import S
```

```
>>> from symbolica.community.spenso import TensorStructure, Representation,  
TensorName
```

```
>>> rep = Representation.euc(3)
```

```
>>> structure = TensorStructure(rep, rep)
```

```
>>> x = S('x')
```

```
>>> structure_with_args = TensorStructure(rep, rep, x)
```

```
>>> T = TensorName("T")
```

```
>>> named_structure = TensorStructure(rep, rep, name=T)
```

**reps\_and\_additional\_args**

**Kind:** Parameter

\*reps\_and\_additional\_args: TensorIndices | builtins.list[Slot]

**name**

**Kind:** Parameter

name: TensorName | builtins.str | Expression | None = None

**Default:** None

Optional tensor name to assign to the structure

**\_\_getitem\_\_**

**Kind:** Method

```
def __getitem__(self, item: builtins.slice) -> builtins.list[Expression |  
builtins.complex | float]:
```

Get expanded indices at the specified range of flattened indices.

Parameters

-----

item : slice

Slice object defining the range of indices

Returns

-----

list of list of int

List of expanded indices

**item**

**Kind:** Parameter

item: builtins.slice

Slice object defining the range of indices

**\_\_getitem\_\_**

**Kind:** Method

```
def __getitem__(self, item: typing.Sequence[builtins.int]) -> Expression |  
builtins.complex | float:
```

Get flattened index associated to this expanded index.

Parameters

-----

item : list of int

Multi-dimensional index coordinates

Returns

-----

int

The flat index

**item**

**Kind:** Parameter

item: typing.Sequence[builtins.int]

Multi-dimensional index coordinates

**\_\_getitem\_\_**

**Kind:** Method

```
def __getitem__(self, item: builtins.int) -> Expression | builtins.complex | float:
```

Get expanded index associated to this flat index.

Parameters

-----

item : int

Flat index into the tensor

Returns

-----

list of int

Multi-dimensional index coordinates

**item**

**Kind:** Parameter

item: builtins.int

Flat index into the tensor

**\_\_call\_\_**

**Kind:** Method

```
def __call__(self, *args: builtins.int | Expression | str, extra_args:
typing.Sequence[Expression | int | str | float | builtins.complex] = []) ->
Expression:
```

Convenience method for creating symbolic expressions.

This is a shorthand for calling ``symbolic(*args, extra_args=extra_args)``.  
Creates a symbolic Expression representing this tensor structure.

Parameters

-----

`*args` : int, str, Symbol, or Expression  
Positional arguments (indices and additional args)  
`extra_args` : list of Expression, optional  
Optional list of additional non-tensorial arguments

Returns

-----

Expression  
A symbolic Expression representing the tensor

Examples

-----

```
>>> structure = TensorStructure(rep, rep, name="T")
>>> expr = structure('mu', 'nu')
```

**args**

**Kind:** Parameter

`*args`: builtins.int | Expression | str

**extra\_args**

**Kind:** Parameter

`extra_args`: typing.Sequence[Expression | int | str | float | builtins.complex] = []

**Default:** []

Optional list of additional non-tensorial arguments

**symbolic**

**Kind:** Method

```
def symbolic(self, *args: builtins.int | Expression | str, extra_args:
typing.Sequence[Expression | int | str | float | builtins.complex] = []) ->
Expression:
```

Create a symbolic expression representing this tensor structure.

Builds a symbolic tensor expression with the specified indices. Arguments can be separated using a semicolon (';') to distinguish between additional arguments and tensor indices.

Parameters

-----

`*args` : int, str, Symbol, Expression, or ';'   
Positional arguments (int, str, Symbol, Expression for indices, ';' for separator)  
`extra_args` : list of Expression, optional  
Optional list of additional non-tensorial arguments



Returns

-----

Expression

A symbolic Expression representing the tensor with indices

Examples

-----

```
>>> import symbolica as sp
>>> from symbolica.community.spenso import TensorStructure, Representation,
TensorName
>>> rep = Representation.euc(3)
>>> T = TensorName("T")
>>> structure = TensorStructure([rep, rep], name=T)
>>> expr = structure.symbolic('mu', 'nu')
>>> x = sp.S('x')
>>> expr = structure.symbolic(x, ';', 'mu', 'nu')
>>> expr = structure.symbolic('mu', 'nu', extra_args=[x])
```

**args**

**Kind:** Parameter

\*args: builtins.int | Expression | str

**extra\_args**

**Kind:** Parameter

extra\_args: typing.Sequence[Expression | int | str | float | builtins.complex] = []

**Default:** []

Optional list of additional non-tensorial arguments

**index**

**Kind:** Method

```
def index(self, *args: builtins.int | Expression | str, extra_args:
typing.Sequence[Expression] = [], cook_indices: builtins.bool = False) ->
TensorIndices:
```

Create an indexed tensor (TensorIndices) from this structure.

Converts this structure template into a concrete indexed tensor by assigning specific abstract indices to each representation slot.

Parameters

-----

\*args : int, str, Symbol, Expression, or ';'

Positional arguments (indices and ';' separator between additional args and indices)

extra\_args : list of Expression, optional

Optional list of additional non-tensorial arguments

cook\_indices : bool, optional

If True, attempt to convert expressions to valid indices

Returns

-----

TensorIndices

A TensorIndices object with concrete index assignments

## Examples

-----

```
>>> import symbolica as sp
>>> from symbolica.community.spenso import TensorStructure, Representation,
TensorName
>>> rep = Representation.cof(3)
>>> T = TensorName("T")
>>> structure = TensorStructure([rep, rep], name=T)
>>> indices = structure.index('mu', 'nu')
>>> x = sp.S('x')
>>> indices = structure.index(x, ';', 'mu', 'nu')
```

## args

**Kind:** Parameter

\*args: builtins.int | Expression | str

## extra\_args

**Kind:** Parameter

extra\_args: typing.Sequence[Expression] = []

**Default:** []

Optional list of additional non-tensorial arguments

## cook\_indices

**Kind:** Parameter

cook\_indices: builtins.bool = False

**Default:** False

If True, attempt to convert expressions to valid indices

[Exhaustive reference](#) · [Source](#)

## ExecutionMode

Execution modes for tensor network evaluation.

Execution modes for tensor network evaluation.

Controls how the tensor network execution engine processes the computational graph.

## Variants

-----

Single : Execute one contraction at a time, useful for debugging

Scalar : Only contract scalar operations, leaving tensor structure intact

All : Execute all possible contractions for complete evaluation

```
class ExecutionMode(enum.Enum):
```

**Requires features:** python\_stubgen

## Introspection scaffold (not a task example)

```
from symbolica.community.spenso import ExecutionMode
help(ExecutionMode)
```

[Exhaustive reference](#) · [Source](#)

## SymbolicParallelism

Policy for Rayon operations that manipulate Symbolica expressions.

Policy for Rayon operations that manipulate Symbolica expressions.

```
class SymbolicParallelism(enum.Enum):
```

**Requires features:** python\_stubgen

### Introspection scaffold (not a task example)

```
from symbolica.community.spenso import SymbolicParallelism
help(SymbolicParallelism)
```

Exhaustive reference · Source

# Canonical package changelog

The following release history is imported from canonical Typst source crates/spenso/CHANGELOG.typ.

All notable changes to this project will be documented in this file.

The format is based on Keep a Changelog, and this project adheres to Semantic Versioning.

## Unreleased

### 0.5.6 - 2026-01-08

#### Fixed

- fix python\_stub\_gen feature
- fix metric initialization and update to symbolica 1.2

#### Other

- update linnet

### 0.5.5 - 2025-12-15

#### Fixed

- fix canonization in presence of duals
- fix sum normalization in network parsing
- fix warnings
- fix overflowing slot order
- fix add\_assign with sparse tensors

#### Other

- Update symbolica to 1.1.0
- update linnet
- add depth to parser, to stop at first mul
- remove more prints
- remove prints
- update linnet
- add readmes
- formatting and add ref for parametric
- update to 0.17 pyo3\_stub\_gen
- update linnet and make classattr to classmethods
- Release spenso-macros 0.3
- update to symbolica 1.0
- update sympolica to 0.20
- Properly precontract scalars
- Add back pyo3-stub-gen-derive
- Updated spenso with minor canonical string update for symbolica f2eb0c1ddcc61c342f2ff9616c2d129141d433dc
- Refactor coefficient conversions to support generic float types
- update to current dev symbolica and add pattern for pslash + m conjugate
- update to linnet 0.13.1
- working canonize (up to functions) and dirac adjoint
- spenso canonize buggy
- robust\_conj but with lots of gamma\_0s

- first try conj
- update python api
- working pow and fun execution
- working pow execution with wrapping
- support for executing pow and functions on tensors
- implement function library trait
- add function generic key

### **0.5.3 - 2025-08-18**

#### **Other**

- update linnet to crates
- all tests pass
- Fix Multiple Heads Bug
- implement ExportNumber for spenso::Complex
- to Atom for ExpandedIndex
- remove printouts
- update linnet
- update linnet
- update linnet
- add infinite loop error test
- update parse problem and add spenso:: to all symbols
- remove expand

### **0.5.2 - 2025-07-15**

#### **Other**

- remove printout

### **0.5.1 - 2025-07-15**

#### **Fixed**

- fix gamma algebra≈

#### **Other**

- allow arbitrary arguments for to dots

## Version information

Save these identifiers with published results and include them in issue reports so that collaborators can reproduce the same software environment.

- **Documentation channel:** latest
- **Documentation route:** /products/spenso/latest/
- **Source revision:** 28275fc5d5154faa43a19dba0233bacd0dafbf1f

## Package versions and enabled features

- gammaloop/gammalooprs — 0.3.4 (no optional features)
- gammaloop/gammaloop-api — 0.1.0 (features: cli)
- gammaloop/gammaloop — 0.3.4 (features: python\_api, python\_abi, python\_stubgen)
- linnet/linnet — 0.17.0 (features: nodestore-vec, serde, bincode, drawing, symbolica)
- linnet/linnet-py — 0.1.0 (features: extension-module, abi3-py310)
- spenso/spenso — 0.6.0 (features: shadowing)
- spenso/spenso-macros — 0.3.2 (features: shadowing)
- spenso/spenso-hep-lib — 0.1.7 (no optional features)
- spenso/spynso3 — 0.1.2 (features: python\_stubgen)
- idenso/idenso — 0.3.0 (features: bincode, reference-cases)
- idenso/idenso — 0.3.0 (features: python, python\_stubgen)
- vakint/vakint — 0.1.2 (no optional features)
- vakint/vakint — 0.1.2 (features: symbolica\_community\_module, python\_stubgen)